

بسمه تعالی



Real Time Embedded Systems

HW3

محمد میثم الهام بخش

۴۰۳۳۰۱۵۲۶

بخش تئوری

(۱) سیستم عامل لینوکس در اغلب موارد به عنوان یک سیستم عامل "نه چندان بلادرنگ" شناخته می‌گردد. در چه شرایطی لینوکس را میتوان برای تسک‌های بلادرنگ استفاده نمود؟ چه محدودیت‌هایی با وجود پتچ‌های PREEMPT_RT هنوز برای لینوکس وجود دارد؟ سناریویی را در نظر بگیرید که سیستم اتوماسیون صنعتی باید حداکثر در مدت زمان ۵ میلی ثانیه به تغییرات یک ورودی واکنش مناسب نشان دهد. برای این سناریو استفاده از لینوکس امبدد با پتچ‌های PREEMPT_RT را توصیه می‌کنید یا یک سیستم عامل بلادرنگ نظیر FreeRTOS ؟ دلیل خود را براساس خصوصیات سیستم شرح دهید.

سیستم‌های بلادرنگ برای انجام وظایف در چارچوب محدودیت‌های زمانی دقیق طراحی شده‌اند و تضمین می‌کنند که عملیات حیاتی در مهلت‌های زمانی تعریف‌شده رخ می‌دهند.¹

۱.۱. لینوکس به عنوان یک سیستم عامل "نه چندان بلادرنگ"

هسته استاندارد لینوکس اصولاً به عنوان یک سیستم عامل بلادرنگ (Real-Time Operating System - RTOS) طراحی نشده است. هدف اصلی آن ارائه عملکرد مناسب برای کاربردهای عمومی و اشتراک منصفانه منابع بین چند وظیفه است. این رویکرد منجر به تأخیرهای متغیر در پاسخ به رویدادها می‌شود که بسته به بار سیستم می‌تواند متفاوت باشد.² این خصوصیت، لینوکس استاندارد را برای کاربردهایی که نیازمند پاسخ‌دهی قطعی و در زمان مقرر هستند، نامناسب می‌سازد.

۱.۲. شرایط استفاده از لینوکس برای تسک‌های بلادرنگ با PREEMPT_RT

برای غلبه بر محدودیت‌های لینوکس استاندارد در زمینه بلادرنگی، مجموعه وصله‌های PREEMPT_RT توسعه داده شده است. این وصله‌ها با ایجاد تغییرات بنیادین در هسته لینوکس، آن را به یک هسته کاملاً قابل قبضه (fully preemptible) تبدیل می‌کنند.¹ این بدان معناست که وظایف با اولویت بالاتر می‌توانند وظایف با اولویت پایین‌تر را، حتی اگر در حال اجرای کد هسته باشند، متوقف کرده و پردازنده را در اختیار بگیرند، مگر اینکه کد در حال اجرا در بخش‌های بسیار کوچکی که با raw_spinlock_t محافظت می‌شوند، قرار داشته باشد.³ این قابلیت، رفتار سیستم را قطعی‌تر کرده و زمان پاسخ به رویدادها را به شدت کاهش می‌دهد.

با اعمال وصله‌های PREEMPT_RT، بدترین حالت تأخیر (worst-case latency) در لینوکس به حدود چند میلی‌ثانیه (single-digit milliseconds) کاهش می‌یابد.³ اهمیت این پیشرفت به حدی بوده است که پس از سال‌ها توسعه، وصله‌های PREEMPT_RT به طور کامل در هسته اصلی لینوکس نسخه ۶.۱۲ (در سپتامبر ۲۰۲۴) ادغام شده و برای معماری‌های RISC-V، x86_64، x86 و ARM۶۴ در دسترس قرار گرفته‌اند.⁴ این ادغام، لینوکس را به گزینه‌ای بسیار جدی‌تر برای کاربردهای بلادرنگ تبدیل کرده است.

۱.۳. محدودیت‌های لینوکس با PREEMPT_RT

علی‌رغم بهبودهای چشمگیر ایجاد شده توسط PREEMPT_RT، هنوز محدودیت‌هایی برای استفاده از لینوکس در سناریوهای بلادرنگ سخت وجود دارد:

۱.۳.۱. کاهش توان عملیاتی (Throughput)

یکی از معاوضه‌های اصلی استفاده از PREEMPT_RT، کاهش احتمالی توان عملیاتی کلی سیستم است. قبضه کردن مکرر وظایف در هسته منجر به افزایش تعداد تعویض زمینه (context switch) می‌شود. هر تعویض زمینه شامل ذخیره وضعیت اجرای فعلی و بارگذاری وضعیت وظیفه جدید است که سربار زمانی خاص خود را دارد. علاوه بر این، وظایف با اولویت پایین‌تر فرصت کمتری برای اجرا پیدا می‌کنند که می‌تواند بر توان عملیاتی کلی تأثیر بگذارد.³

۱.۳.۲. بخش‌های غیرقابل قبضه (Non-preemptible)

همانطور که اشاره شد، حتی با PREEMPT_RT، بخش‌های بسیار کوچکی از کد هسته که با raw_spinlock_t محافظت می‌شوند، غیرقابل قبضه باقی می‌مانند.³ اگرچه این بخش‌ها به حداقل ممکن کاهش یافته‌اند، اما همچنان می‌توانند منبعی برای تأخیرهای پیش‌بینی نشده باشند.

۱.۳.۳. پیچیدگی درایورهای سخت‌افزاری و پیکربندی

عملکرد بلادرنگ به شدت تحت تأثیر درایورهای سخت‌افزاری است. وصله‌های PREEMPT_RT برای هسته وانیلی لینوکس نوشته شده‌اند و درایورهایی که بخشی از هسته اصلی نیستند (مانند درایورهای خاص برخی BSPها) ممکن است با این وصله‌ها سازگار نباشند یا اعمال وصله‌ها روی آن‌ها با خطا مواجه شود.

برای دستیابی به عملکرد بلادرنگ مطلوب، ممکن است لازم باشد برخی درایورها، به ویژه درایورهای مرتبط با نمایشگر/گرافیک (مانند (Vivante، DRM، FB و همچنین قابلیت‌های مدیریت توان یا تنظیم فرکانس CPU غیرفعال شوند، زیرا این موارد می‌توانند باعث ایجاد تأخیر شوند. ۵ پیکربندی‌های دیگری مانند فعال‌سازی تایمر با دقت بالا (high resolution timer) به حداقل رساندن پیام‌های کنسول (با آرگومان quiet یا تنظیم سطح kernel.printk و غیرفعال کردن تخصیص بیش از حد حافظه (vm.overcommit_memory = 2) نیز برای کاهش تأخیر توصیه می‌شود.

۱.۳.۴. ماهیت تجربی و وضعیت اعتبارسنجی (به خصوص در پلتفرم‌های خاص)

برخی تولیدکنندگان سخت‌افزار، مانند BSP، Toradex و ایمپج‌های دارای PREEMPT_RT را در وضعیت تجربی (experimental) منتشر می‌کنند و آن‌ها را به طور کامل اعتبارسنجی نمی‌کنند. در چنین مواردی، مسئولیت اعتبارسنجی عملکرد بلادرنگ برای کاربرد خاص بر عهده کاربر است و تولیدکننده تضمینی برای عملکرد یا رفع اشکال ارائه نمی‌دهد.^۵ این موضوع نشان می‌دهد که اگرچه PREEMPT_RT در هسته اصلی ادغام شده، پیاده‌سازی و اعتبارسنجی آن روی پلتفرم‌های خاص همچنان نیازمند دقت و تلاش است.

۱.۳.۵. عدم تضمین بلادرنگی سخت (Hard Real-Time)

مهم است که بدانیم لینوکس با PREEMPT_RT هنوز یک سیستم‌عامل بلادرنگ سخت (Hard RTOS) به معنای کلاسیک آن نیست. RTOS‌های سنتی از ابتدا با هدف قطعیت و پیش‌بینی‌پذیری مطلق طراحی شده‌اند و اغلب تحت آزمایش‌ها و گواهی‌نامه‌های سخت‌گیرانه‌ای برای صنایع حیاتی (پزشکی، هسته‌ای، هوافضا) قرار می‌گیرند. در حالی که PREEMPT_RT عملکرد بلادرنگ لینوکس را به طور قابل توجهی بهبود می‌بخشد، برای سیستم‌هایی که کوچکترین خطا در رعایت مهلت زمانی منجر به فاجعه می‌شود و نیازمند گواهی‌نامه‌های بسیار دقیق هستند، یک RTOS اختصاصی ممکن است همچنان گزینه ارجح یا الزامی باشد.^۶

۱.۴. سناریوی اتوماسیون صنعتی (پاسخ‌دهی ۵ میلی‌ثانیه)

۱.۴.۱. تحلیل نیازمندی‌ها

مهلت زمانی ۵ میلی‌ثانیه یک نیازمندی بلادرنگ قطعی (firm real-time) محسوب می‌شود. در این نوع سیستم‌ها، از دست دادن مهلت زمانی به عنوان یک شکست سیستم تلقی می‌شود، اما لزوماً عواقب فاجعه‌بار (مانند سیستم‌های بلادرنگ سخت) را به دنبال ندارد، هرچند می‌تواند منجر به مشکلات تولید شود.^۲

۱.۴.۲. لینوکس امبدد با PREEMPT_RT

- توانمندی‌ها: لینوکس با PREEMPT_RT قادر است به تأخیرهایی در حد تک‌رقمی میلی‌ثانیه دست یابد.³ اندازه‌گیری‌های انجام شده روی بردهای ARM با PREEMPT_RT نشان داده‌اند که حد بالای تأخیر می‌تواند در حدود ۱۵۰ میکروثانیه باشد و اکثر تأخیرها زیر ۵۰ میکروثانیه هستند.⁷ نتایج cyclicttest روی Raspberry Pi با PREEMPT_RT میانگین تأخیر حدود ۱۰ میکروثانیه و بدترین حالت حدود ۸۳ میکروثانیه را نشان داده است.⁷ همچنین، سیستم‌های صوتی با PREEMPT_RT می‌توانند به راحتی تأخیرهای زیر ۵ میلی‌ثانیه را مدیریت کنند.⁸ جیتر (تغییرات در زمان پاسخ) برای وظایف متناوب حتی تحت بار می‌تواند صفر باشد.⁷
- ملاحظات: کاهش توان عملیاتی³ و پیچیدگی‌های مربوط به درایور و پیکربندی⁵ باید در نظر گرفته شوند.

۱.۴.۳. سیستم عامل بلادرنگ (مانند FreeRTOS)

- توانمندی‌ها: FreeRTOS و سیستم‌عامل‌های مشابه به طور خاص برای قطعیت و پیش‌بینی‌پذیری در اجرای وظایف طراحی شده‌اند.² این سیستم‌عامل‌ها تحت آزمایش‌های دقیق قرار گرفته و برای صنایع حیاتی گواهی‌نامه دریافت می‌کنند.⁶ پاسخ‌دهی با تأخیر کم در آن‌ها تضمین شده است.²
- ملاحظات: این سیستم‌عامل‌ها اغلب تخصصی بوده و ممکن است منابع بیشتری مصرف کنند. همچنین، عملکرد آن‌ها فراتر از وظایف بلادرنگ محدود است که می‌تواند در سناریوهایی که نیاز به اجرای برنامه‌های متنوع وجود دارد، یک نقطه ضعف باشد.⁶

۱.۴.۴. توصیه و توجیه

در حالی که یک RTOS مانند FreeRTOS قطعیت بالاتری را به دلیل فلسفه طراحی خود ارائه می‌دهد، برای یک مهلت زمانی ۵ میلی‌ثانیه که در دسته بلادرنگ قطعی (firm real-time) قرار می‌گیرد، لینوکس با PREEMPT_RT معمولاً به اندازه کافی قطعی و پاسخگو است. انتخاب بین این دو تنها به تأخیر خام بستگی ندارد، بلکه یک معاوضه چندوجهی شامل پیچیدگی سیستم، اکوسیستم توسعه، نیازمندی‌های غیربلادرنگ و اهمیت واقعی مهلت زمانی است. اگر از دست رفتن مهلت زمانی ۵ میلی‌ثانیه عواقب ایمنی فاجعه‌بار داشته باشد و گواهی‌نامه‌های سخت‌گیرانه‌ای مورد نیاز باشد، آنگاه FreeRTOS گزینه مطمئن‌تری خواهد بود. اما برای اکثر کاربردهای اتوماسیون صنعتی با این مشخصات، لینوکس با PREEMPT_RT راه‌حلی توانمند و همه‌کاره‌تر ارائه می‌دهد. پیشرفت‌های مداوم و ادغام PREEMPT_RT در هسته اصلی، مرزهای بین لینوکس و RTOS‌های سنتی

را کم‌رنگ‌تر کرده و لینوکس را به گزینه‌ای جذاب برای طیف وسیع‌تری از کاربردهای بلادرنگ تبدیل می‌کند، هرچند فلسفه‌های طراحی بنیادین آن‌ها همچنان برای سناریوهای بلادرنگ بسیار سخت، تمایز ایجاد می‌کند.

۲) رفتار و موارد استفاده سیاست‌های زمان‌بندی SCHED_FIFO, SCHED_RR در لینوکس را با یکدیگر مقایسه کنید. برای یک سیستم داده‌برداری با خصوصیات بلادرنگ نرم، به چه صورت سیاست زمان‌بندی مناسب را انتخاب خواهید کرد؟

لینوکس چندین سیاست زمان‌بندی را برای کنترل نحوه تخصیص زمان CPU به نخ‌ها (threads) و فرآیندها ارائه می‌دهد. برای کاربردهای بلادرنگ، دو سیاست اصلی مبتنی بر استاندارد POSIX وجود دارد: SCHED_FIFO و ¹SCHED_RR

۲.۱. مقدمه‌ای بر سیاست‌های زمان‌بندی بلادرنگ در لینوکس

سیاست‌های زمان‌بندی بلادرنگ در لینوکس (SCHED_FIFO و SCHED_RR) از اولویت‌های ثابت (static priorities) در بازه ۱ (پایین‌ترین) تا ۹۹ (بالا‌ترین) استفاده می‌کنند. این نخ‌های بلادرنگ همیشه بر نخ‌هایی که با سیاست‌های زمان‌بندی عادی مانند SCHED_OTHER (یا SCHED_NORMAL) اجرا می‌شوند، اولویت دارند.¹ سیاست SCHED_OTHER از اولویت‌های پویا و مقدار nice برای تنظیم اولویت استفاده می‌کند.

۲.۲. سیاست SCHED_FIFO (First-In, First-Out)

- رفتار: یک نخ با سیاست SCHED_FIFO تا زمانی که یکی از شرایط زیر رخ دهد، به اجرای خود ادامه می‌دهد:¹
 - نخ به خواب برود یا منتظر یک رویداد شود (مثلاً عملیات I/O).
 - نخ به طور داوطلبانه پردازنده را واگذار کند (با فراخوانی sched_yield()).
 - یک نخ بلادرنگ با اولویت بالاتر آماده اجرا شود و آن را قبضه کند. در سیاست SCHED_FIFO، هیچ برش زمانی (time slicing) برای نخ‌هایی که اولویت یکسانی دارند، وجود ندارد. اگر چندین نخ با اولویت یکسان آماده اجرا باشند، نخ‌ی که زودتر وارد صف آماده شده، اول اجرا می‌شود و تا زمانی که یکی از شرایط فوق رخ ندهد، به اجرای خود ادامه می‌دهد.¹
- موارد استفاده: SCHED_FIFO برای وظایف بلادرنگی مناسب است که باید به محض آماده شدن، با کمترین تأخیر ممکن اجرا شده و تا حد امکان بدون وقفه (از جانب نخ‌های با اولویت برابر یا پایین‌تر) به کار خود ادامه دهند.¹ این سیاست زمانی ایده‌آل است که یک وظیفه حیاتی منفرد وجود دارد و ترتیب ورود به

صف برای نخ‌های هم‌اولویت، ترتیب اجرای مطلوب است.

۲.۳. سیاست SCHED_RR (Round-Robin)

- رفتار: سیاست SCHED_RR بسیار شبیه به SCHED_FIFO است، با این تفاوت کلیدی که برای نخ‌های با اولویت یکسان، از زمان‌بندی نوبتی (Round-Robin) با یک کوانتوم یا برش زمانی (time slice/quantum) مشخص استفاده می‌کند.¹ هر نخ SCHED_RR یک برش زمانی برای اجرا دریافت می‌کند. اگر نخ قبل از اتمام برش زمانی خود مسدود نشود یا توسط نخ با اولویت بالاتر قبضه نگردد، پس از اتمام برش زمانی، به انتهای صف نخ‌های آماده با همان اولویت منتقل شده و نخ بعدی در صف اجرا می‌شود.¹ اگر تنها یک نخ با اولویت مشخص وجود داشته باشد، رفتار SCHED_RR دقیقاً مشابه SCHED_FIFO خواهد بود.¹³
- تعیین کوانتوم زمانی: طول کوانتوم زمانی برای SCHED_RR توسط سیستم تعیین می‌شود و می‌توان آن را با استفاده از فراخوان سیستمی sched_rr_get_interval (۲) بازیابی کرد.¹⁴
- موارد استفاده: SCHED_RR زمانی مناسب است که چندین وظیفه بلادرنگ با اولویت یکسان وجود دارند و نیاز است که به طور منصفانه به CPU دسترسی داشته باشند تا از گرسنگی (starvation) یکی از آن‌ها توسط دیگری جلوگیری شود.¹ این سیاست برای سناریوهایی که چندین وظیفه مهم باید به طور همزمان پیشرفت کنند، مفید است.

۲.۴. مقایسه SCHED_RR و SCHED_FIFO

- شباهت‌ها: هر دو سیاست بلادرنگ هستند، از اولویت‌های ثابت در بازه ۱ تا ۹۹ استفاده می‌کنند و نخ‌های SCHED_OTHER را قبضه می‌کنند.¹ نخ‌های تحت هر دو سیاست تا زمانی که مسدود شوند، پردازنده را واگذار کنند یا توسط نخ با اولویت بالاتر قبضه شوند، به اجرا ادامه می‌دهند.¹¹
- تفاوت اصلی: تفاوت عمده در نحوه مدیریت نخ‌های با اولویت یکسان است. SCHED_FIFO به نخ‌ی که اول وارد صف شده اجازه می‌دهد تا زمانی که بخواهد اجرا شود (بدون برش زمانی)، در حالی که SCHED_RR بین نخ‌های هم‌اولویت با استفاده از برش زمانی به اشتراک می‌گذارد.¹

• جدول مقایسه:

ویژگی	SCHED_FIFO	SCHED_RR
اولویت	ثابت (۹۹-۱)	ثابت (۹۹-۱)
قبضه‌پذیری	بله، توسط نخ با اولویت بالاتر	بله، توسط نخ با اولویت بالاتر
برش زمانی	خیر (برای نخ‌های با اولویت یکسان)	بله (برای نخ‌های با اولویت یکسان)
رفتار با اولویت یکسان	نخی که اول وارد صف شده، اول اجرا می‌شود و تا زمان مسدود شدن به اجرا ادامه می‌دهد.	نخ‌ها به نوبت و با برش زمانی مشخص اجرا می‌شوند (Round-Robin).
گرسنگی	ریسک گرسنگی برای نخ‌های با اولویت پایین‌تر یا برابر (اگر نخ فعلی CPU را رها نکند).	ریسک گرسنگی برای نخ‌های با اولویت پایین‌تر؛ اشتراک عادلانه‌تر CPU بین نخ‌های هم‌اولویت.

۲.۵. انتخاب سیاست زمان‌بندی برای سیستم داده‌برداری بلادرنگ نرم

یک سیستم داده‌برداری با خصوصیات بلادرنگ نرم، سیستمی است که در آن رعایت مهلت‌های زمانی مهم است، اما از دست دادن یک مهلت زمانی منجر به تنزل عملکرد می‌شود نه یک شکست فاجعه‌بار.¹⁵ چنین سیستمی ممکن است شامل وظایف متعددی برای دریافت داده، پردازش اولیه و ثبت آن‌ها باشد.

تحلیل و توصیه:

- اگر در سیستم داده‌برداری، یک نخ اصلی و بسیار حیاتی برای دریافت داده وجود دارد که باید به محض در دسترس قرار گرفتن داده‌ها و با کمترین جیتر ممکن اجرا شود، سیاست SCHED_FIFO با اولویت بالا اغلب انتخاب بهتری است. این تضمین می‌کند که این نخ، سایر کارها را قبضه کرده و تا زمان تکمیل (یا تا زمانی که به طور داوطلبانه CPU را رها کند یا مسدود شود) اجرا گردد.¹⁵
- اگر چندین وظیفه بلادرنگ همزمان با اهمیت مشابه وجود دارند (مثلاً پردازش همزمان جریان داده از چند سنسور)، SCHED_RR با اختصاص برش زمانی به هر یک، از گرسنگی یکی توسط دیگری جلوگیری کرده و پیشرفت منصفانه همه آن‌ها را تضمین می‌کند. ماهیت "نرم" بلادرنگی در اینجا به این معنی است که قبضه شدن گاه‌به‌گاه توسط برش زمانی SCHED_RR ممکن است قابل قبول باشد، اگر این امر تضمین

کند که تمام وظایف مهم پیشرفت می‌کنند.

به طور کلی، در سیستم‌های بلادرنگ نرم، انتخاب بین SCHED_FIFO و SCHED_RR به تعداد نخ‌های حیاتی و همزمان و اینکه آیا تکمیل قطعی یک نخ منفرد یا پیشرفت منصفانه چندین نخ در اولویت است، بستگی دارد. SCHED_FIFO به دلیل سادگی و سربار کمتر (به دلیل عدم نیاز به مدیریت برش زمانی) معمولاً در مواردی که برش زمانی به طور خاص مورد نیاز نیست، ترجیح داده می‌شود.¹¹ برای یک سیستم داده‌برداری که ممکن است یک نخ اصلی برای دریافت داده و نخ‌های دیگری برای پردازش یا ذخیره‌سازی داشته باشد، می‌توان نخ دریافت‌کننده داده را با SCHED_FIFO و اولویت بسیار بالا تنظیم کرد و نخ‌های پردازشی (اگر چندتایی و هم‌اهمیت باشند) را با SCHED_RR و اولویت کمی پایین‌تر یا حتی با SCHED_FIFO و اولویت‌های متفاوت (اگر سلسله مراتب مشخصی بین آن‌ها وجود دارد) اجرا نمود. انتخاب بهینه نیازمند تحلیل دقیق جریان داده و نیازمندی‌های زمانی هر وظیفه در سیستم است.

۳) سیستم های امبدد اغلب از حافظه فلش استفاده میکنند. دو فایل سیستم رایج لینوکس در دستگاه های امبدد را با یکدیگر مقایسه کنید. (برای مثال JSFS2, ext4) تفاوت های کلیدی این دو فایل سیستم از نظر میزان سایش، زمان نصب و اطمینان پذیری بعد از قطع تغذیه را با یکدیگر مقایسه نمایید.

سیستم های نهفته اغلب از حافظه فلش برای ذخیره سازی استفاده می کنند. حافظه های فلش (مانند NAND و NOR) دارای مشخصات منحصر به فردی هستند که فایل سیستم های سنتی طراحی شده برای دیسک های گردان را برای آنها نامناسب می سازد.

۳.۱. مقدمه ای بر حافظه فلش و چالش های آن در سیستم های نهفته

حافظه های فلش از واحدهایی به نام "بلاک" تشکیل شده اند که قبل از نوشتن مجدد باید به طور کامل پاک شوند (erase-before-write). هر بلاک تعداد محدودی چرخه پاک کردن را تحمل می کند (معمولاً ۱۰۰۰ تا ۱۰۰۰۰ برای MLC NAND و ۱۰۰ هزار تا ۱ میلیون برای SLC NAND/NOR) و پس از آن فرسوده می شود. این امر نیازمند مکانیزم های "توزیع سایش" (wear leveling) برای توزیع یکنواخت عملیات نوشتن در سراسر حافظه است تا از فرسودگی زودهنگام بخشی از حافظه جلوگیری شود. همچنین، حافظه های NAND ممکن است دارای "بلوک های خراب" (bad blocks) از کارخانه باشند که باید توسط نرم افزار مدیریت شوند.¹⁷

دو دسته اصلی حافظه فلش در سیستم های نهفته استفاده می شوند:

- حافظه فلش خام (Raw NAND/NOR): این حافظه ها مستقیماً به گذرگاه حافظه سیستم متصل می شوند و سیستم عامل (از طریق لایه MTD - Memory Technology Device در لینوکس) مسئولیت کامل مدیریت سایش، بلوک های خراب و عملیات سطح پایین فلش را بر عهده دارد. فایل سیستم هایی مانند JFFS2 و UBIFS برای این نوع حافظه ها طراحی شده اند.¹⁸
- دستگاه های دارای لایه ترجمه فلش (FTL - Flash Translation Layer): این شامل کارت های حافظه (SD, microSD) و حافظه های eMMC می شود. این دستگاه ها دارای یک کنترلر داخلی هستند که عملیات سطح پایین فلش، توزیع سایش و مدیریت بلوک های خراب را انجام داده و یک رابط بلاک استاندارد (مانند دیسک سخت) به سیستم عامل ارائه می دهند. فایل سیستم های بلاک مانند ext4 یا FAT روی این دستگاه ها استفاده می شوند.¹⁸

۳.۲. مقایسه دو فایل سیستم رایج (JFFS2 و ext4)

در این بخش، دو فایل سیستم رایج، JFFS2 (به عنوان نماینده فایل سیستم های مخصوص فلش خام) و ext4 (به عنوان نماینده فایل سیستم های بلاک روی دستگاه های FTL)، مقایسه می شوند.

۳.۲.۱. JFFS2 (Journaling Flash File System version 2)

- معماری: JFFS2 یک فایل سیستم لاگ-ساختار (log-structured) است که به طور خاص برای کار با دستگاه های MTD خام (NOR و NAND) طراحی شده است.¹⁸ داده ها و فراداده ها (فراداده ها شامل اطلاعاتی مانند نوع فایل، مجوزهای دسترسی و غیره) به صورت "گره" (node) به طور متوالی روی حافظه فلش نوشته می شوند.¹⁹ هر گره نشان دهنده یک فایل یا بخشی از یک فایل است.
- سایش JFFS2: (Wear Leveling) مکانیزم توزیع سایش نرم افزاری را پیاده سازی می کند. هنگامی که داده ها به روزرسانی می شوند، نسخه جدید گره در یک بلاک جدید با فضای خالی نوشته می شود و گره قدیمی نامعتبر تلقی می گردد. یک فرآیند "جمع آوری زباله" (garbage collection) در پس زمینه اجرا می شود تا بلاک هایی که حاوی گره های نامعتبر (کثیف - dirty) هستند را پاک کرده و فضای آن ها را بازپس گیرد. برای توزیع سایش، این فرآیند گاهی اوقات بلاک های تمیز (clean - حاوی گره های معتبر) را نیز انتخاب کرده، داده های معتبر آن ها را به مکان دیگری منتقل و سپس بلاک را پاک می کند.¹⁸ توزیع سایش در JFFS2 به صورت محلی در سطح پارتیشن انجام می شود.¹⁸
- زمان نصب: (Mount Time) در زمان نصب، JFFS2 باید کل پارتیشن را اسکن کند تا نقشه درون-حافظه ای گره های معتبر را برای هر فایل بازسازی کند. این فرآیند می تواند برای پارتیشن های بزرگ بسیار زمان بر باشد.¹⁷ فعال کردن گزینه CONFIG_JFFS2_SUMMARY در پیکربندی هسته می تواند با ذخیره خلاصه ای از این نقشه روی خود فلش، زمان نصب را به طور قابل توجهی بهبود بخشد.¹⁸
- اطمینان پذیری پس از قطع تغذیه: JFFS2 به گونه ای طراحی شده است که در برابر قطعی ناگهانی برق مقاوم باشد. ماهیت لاگ-ساختار و ژورنالینگ ذاتی آن به حفظ یکپارچگی فایل سیستم کمک می کند.¹⁸ پس از قطعی برق، با اسکن مجدد، وضعیت سازگار فایل سیستم بازیابی می شود.
- سایر ویژگی ها: JFFS2 از فشرده سازی درجا (on-the-fly compression) با الگوریتم هایی مانند Zlib (فشرده سازی بهتر) و LZO (عملکرد سریع تر) پشتیبانی می کند.¹⁷ همچنین قادر به مدیریت بلوک های خراب ذاتی حافظه های NAND است.¹⁸

۳.۲.۲. ext4 (Fourth Extended Filesystem)

- معماری: ext4 یک فایل سیستم ژورنالینگ است که اساساً برای دستگاه‌های بلاک طراحی شده است. در سیستم‌های نهفته، معمولاً روی دستگاه‌های دارای FTL مانند کارت‌های SD و حافظه‌های eMMC استفاده می‌شود.¹⁹
- سایش: **(Wear Leveling)** خودِ فایل سیستم ext4 هیچ مکانیزم توزیع سایشی را پیاده‌سازی نمی‌کند. این وظیفه کاملاً بر عهده FTL سخت‌افزاری دستگاه زیرین (مانند کنترلر کارت SD یا eMMC) است.¹⁸ کیفیت و کارایی الگوریتم‌های توزیع سایش FTL می‌تواند بین دستگاه‌های مختلف متفاوت باشد و اغلب برای کاربر شفاف نیست.¹⁸
- زمان نصب: **(Mount Time)** به طور کلی، زمان نصب ext4 روی پارتیشن‌های بزرگ، به خصوص پس از یک خاموشی تمیز (clean shutdown)، سریع‌تر از JFFS2 است. پس از یک خاموشی ناگهانی (unclean shutdown)، فرآیند پخش مجدد ژورنال (journal replay) برای بازگرداندن فایل سیستم به حالت سازگار، معمولاً بسیار سریع است.²⁵
- اطمینان‌پذیری پس از قطع تغذیه: ext4 از ژورنالینگ برای تضمین یکپارچگی فراداده‌ها و قابلیت بازیابی پس از خرابی یا قطعی برق استفاده می‌کند. ext4¹⁹ سه حالت ژورنالینگ اصلی ارائه می‌دهد:
 - data=journal: امن‌ترین حالت که هم داده‌ها و هم فراداده‌ها را قبل از نوشتن در فایل سیستم اصلی، در ژورنال ثبت می‌کند. این حالت بیشترین سربار عملکردی را دارد.²⁵
 - data=ordered (پیش‌فرض): فقط فراداده‌ها در ژورنال ثبت می‌شوند، اما نوشتن داده‌ها در فایل سیستم اصلی قبل از ثبت فراداده‌های مرتبط با آن‌ها در ژورنال انجام می‌شود. این حالت تعادل خوبی بین امنیت و عملکرد ارائه می‌دهد.²⁵
 - data=writeback: فقط فراداده‌ها ژورنال می‌شوند و هیچ ترتیبی بین نوشتن داده‌ها و فراداده‌ها تضمین نمی‌شود. این حالت سریع‌ترین عملکرد را دارد اما کمترین امنیت را در برابر از دست رفتن داده‌ها پس از قطعی برق ارائه می‌دهد.²⁵ برای کاربردهای نهفته که پایداری در برابر قطعی برق مهم است، استفاده از حالت data=journal یا data=ordered همراه با گزینه‌های نصب مانند commit (برای کاهش فاصله زمانی ثبت ژورنال) یا sync (برای نوشتن همزمان داده‌ها روی دیسک) توصیه می‌شود.²⁵
- سایر ویژگی‌ها: ext4 یک فایل سیستم بالغ، پرکاربرد و با امکانات فراوان است. با این حال، به طور خاص برای حافظه‌های فلش خام طراحی نشده است.

۳.۲.۳. جدول مقایسه JFFS2 و ext4 (برای حافظه فلش)

ویژگی	JFFS2	Ext4 (روی دستگاه FTL)
نوع حافظه هدف	حافظه فلش خام (NOR/NAND) از طریق لایه MTD	دستگاه‌های بلاک (مانند eMMC، کارت SD) که دارای FTL داخلی هستند.
مکانیزم سایش	نرم‌افزاری داخلی، در سطح پارتیشن. مدیریت مستقیم چرخه‌های پاک کردن و توزیع نوشتن‌ها.	وابسته به FTL سخت‌افزاری دستگاه. فایل سیستم کنترلی بر آن ندارد.
زمان نصب (اولیه/بزرگ)	کند به دلیل نیاز به اسکن کامل پارتیشن برای بازسازی ساختار فایل سیستم در حافظه. با گزینه SUMMARY بهبود می‌یابد.	نسبتاً سریع، به خصوص پس از خاموشی تمیز.
زمان نصب (پس از قطعی برق)	مشابه نصب اولیه (نیاز به اسکن و بازسازی برای تضمین یکپارچگی).	سریع، از طریق پخش مجدد ژورنال برای بازگرداندن فایل سیستم به حالت سازگار.
اطمینان‌پذیری قطعی برق	بالا؛ طراحی لاگ-ساختار و ژورنالینگ ذاتی آن برای مقاومت در برابر قطعی برق بهینه شده است.	بالا با تنظیمات ژورنالینگ مناسب (مانند data=journal یا data=ordered). ژورنال از خرابی فراداده جلوگیری می‌کند.
فشرده‌سازی	پشتیبانی از فشرده‌سازی درجا (on-the-fly) برای داده‌ها و فراداده‌ها (مثلاً، zlib، lz0).	به طور ذاتی در سطح فایل سیستم پشتیبانی نمی‌کند (نیاز به راه‌حل‌های لایه بالاتر دارد).
مدیریت بلوک‌های خراب	به طور داخلی بلوک‌های خراب را شناسایی و از آن‌ها اجتناب می‌کند.	وابسته به FTL دستگاه برای مدیریت بلوک‌های خراب. فایل سیستم از وجود آن‌ها بی‌اطلاع است.

انتخاب بین یک فایل سیستم آگاه از فلش خام مانند JFFS2 (یا جایگزین مدرن‌تر آن UBIFS) و یک فایل سیستم بلاک سنتی مانند ext4 روی یک دستگاه FTL، یک تصمیم معماری اساسی در سیستم‌های لینوکس نهفته است. این انتخاب، معاوضه‌ای است بین کنترل مستقیم و مدیریت بهینه فلش (که JFFS2/UBIFS ارائه می‌دهند) و راحتی استفاده از رابط بلاک استاندارد و رفتار بالقوه غیرشفاف FTL‌های سخت‌افزاری (که با ext4 همراه

است). در حالی که ژورنالینگ ext4 بازیابی خوبی را پس از قطعی برق برای فراداده‌ها فراهم می‌کند، اتکای آن به FTL برای توزیع سایش و مدیریت بلوک‌های خراب به این معنی است که قابلیت اطمینان بلندمدت آن روی حافظه فلش به کیفیت پیاده‌سازی FTL بستگی دارد، که اغلب برای توسعه‌دهنده یک "جعبه سیاه" است. JFFS2 مدیریت شفاف‌تری بر فلش ارائه می‌دهد اما به قیمت عملکرد، به خصوص زمان نصب روی دستگاه‌های بزرگتر. برای حافظه‌های NAND خام بزرگتر، UBIFS اغلب جایگزین بهتری برای JFFS2 محسوب می‌شود، زیرا مشکلات مقیاس‌پذیری JFFS2 را تا حد زیادی حل کرده و عملکرد بهتری ارائه می‌دهد.¹⁷

۴) چه خطرات و آسیب پذیری هایی سیستم های لینوکس امبدد متصل به اینترنت را تهدید میکند؟ یک نمونه را مثال بزنید و برای آن راه حلی ارائه کنید که مصالحه و تعادلی مناسبی بین امنیت و عملکرد ارائه دهد.

اتصال سیستم های لینوکس نهفته به اینترنت، آن ها را در معرض طیف وسیعی از تهدیدات و آسیب پذیری های امنیتی قرار می دهد. درک این خطرات و اتخاذ تدابیر مناسب برای مقابله با آن ها برای حفظ امنیت و پایداری این سیستم ها حیاتی است.

۴.۱. سطح حمله گسترده سیستم های نهفته متصل به شبکه

سیستم های نهفته متصل به اینترنت به دلیل دسترسی پذیری از طریق شبکه، دارای سطح حمله گسترده ای هستند.²⁸ این سیستم ها اغلب دارای منابع محدود، چرخه های به روزرسانی طولانی تر و گاهی اوقات نرم افزارهای قدیمی هستند که آن ها را به اهداف جذابی برای مهاجمان تبدیل می کند. لیست ۱۰ آسیب پذیری برتر اینترنت اشیاء (IoT) که توسط OWASP (Open Web Application Security Project) منتشر شده، به طور گسترده ای برای این سیستم ها قابل اعمال است.²⁹

۴.۲. آسیب پذیری های رایج (بر اساس OWASP IoT Top)

بسیاری از آسیب پذیری های رایج در سیستم های لینوکس نهفته متصل به اینترنت ریشه در فشار برای عرضه سریع محصول، الزامات محصول که راحتی را بر امنیت ترجیح می دهند، و آموزش ناکافی تیم های توسعه در مورد شیوه های توسعه امن دارد.²⁹ برخی از این آسیب پذیری ها عبارتند از:

۱. رمزهای عبور ضعیف، قابل حدس، یا کدگذاری شده (Hardcoded): استفاده از اعتبارنامه های پیش فرض، ساده یا غیرقابل تغییر، از جمله درب های پشتی (backdoors) در فریمور یا نرم افزار کلاینت.²⁹
۲. سرویس های شبکه ناامن: اجرای سرویس های شبکه ای غیرضروری یا ناامن روی خود دستگاه، به ویژه آن هایی که در معرض اینترنت هستند (مانند SSH یا وب سرورهای باقی مانده از مرحله توسعه).²⁹
۳. رابط های اکوسیستم ناامن: آسیب پذیری در رابط های وب، API های پشتیبان، سرویس های ابری و برنامه های موبایل مرتبط با دستگاه، شامل فقدان احراز هویت یا مجوزدهی مناسب، رمزنگاری ضعیف یا عدم رمزنگاری، و فیلتر نکردن ورودی/خروجی ها.²⁹
۴. فقدان مکانیزم به روزرسانی امن: عدم توانایی در به روزرسانی امن دستگاه، شامل عدم اعتبارسنجی فریمور روی دستگاه، تحویل ناامن (بدون رمزنگاری در حین انتقال)، عدم وجود مکانیزم های ضد بازگشت به عقب

(anti-rollback) و عدم اطلاع‌رسانی تغییرات امنیتی ناشی از به‌روزرسانی‌ها.²⁹

۵. استفاده از اجزای ناامن یا قدیمی: استفاده از کتابخانه‌ها، سیستم‌عامل‌ها یا اجزای سخت‌افزاری منسوخ یا

دارای آسیب‌پذیری‌های شناخته‌شده، یا استفاده از اجزا از یک زنجیره تأمین به خطر افتاده.²⁹

۶. حفاظت ناکافی از حریم خصوصی: ذخیره یا انتقال ناامن اطلاعات شخصی کاربران بدون رضایت یا با

حفاظت ناکافی.²⁹

۷. انتقال و ذخیره‌سازی داده ناامن: عدم استفاده از رمزنگاری یا کنترل دسترسی برای داده‌های حساس در

حالت سکون (at rest)، در حال انتقال (in transit) یا در حین پردازش.²⁹

۸. فقدان مدیریت دستگاه: پشتیبانی امنیتی ناکافی برای دستگاه‌های مستقر شده، شامل مدیریت دارایی،

مدیریت به‌روزرسانی، از رده خارج کردن امن، نظارت بر سیستم و قابلیت‌های پاسخ به حوادث.²⁹

۹. تنظیمات پیش‌فرض ناامن: عرضه دستگاه‌ها با تنظیمات پیش‌فرض ناامن (مانند مجوزهای دسترسی

نامناسب فایل سیستم، اجرای سرویس‌ها با دسترسی root) یا عدم امکان امن‌تر کردن سیستم توسط

اپراتورها.²⁹

۱۰. فقدان سخت‌سازی فیزیکی: عدم وجود اقدامات سخت‌سازی فیزیکی کافی، که به مهاجمان اجازه می‌دهد با

دسترسی فیزیکی به اطلاعات حساس دست یابند یا کنترل محلی دستگاه را به دست گیرند (مانند

پورت‌های دیباگ JTAG یا UART بدون محافظت).²⁹

۴.۳. مثال آسیب‌پذیری: CVE-۲۰۲۵-۳۲۴۳۳ (اجرای کد از راه دور بدون احراز هویت در SSH سرور

(Erlang/OTP)

این آسیب‌پذیری که در ۱۶ آوریل ۲۰۲۵ افشا شد، یک نقص حیاتی با امتیاز CVSS برابر با ۱۰.۰ در مؤلفه سرور

SSH پلتفرم Erlang/OTP است.³¹ سیستم‌های لینوکس نهفته که از سرور SSH داخلی Erlang/OTP استفاده

می‌کنند (مثلاً در تجهیزات مخابراتی، اینترنت اشیاء، پلتفرم‌های ابری و پایگاه‌های داده) در معرض خطر هستند.

• شرح آسیب‌پذیری و مکانیسم حمله: نقص در نحوه مدیریت پیام‌های دسته‌دهی، انواع پیام با شناسه ۸۰ یا

بیشتر برای مراحل پس از احراز هویت کاربر رزرو شده‌اند. با این حال، سرور Erlang/OTP آسیب‌پذیر این

قانون را به درستی اجرا نمی‌کند و به کلاینت اجازه می‌دهد چنین پیام‌هایی را قبل از تکمیل فرآیند احراز

هویت ارسال کند. مهاجم می‌تواند با ارسال یک درخواست باز کردن کانال و بلافاصله پس از آن یک

درخواست کانال حاوی یک دستور مخرب (payload)، قبل از ارائه هرگونه اعتبارنامه، کد دلخواه را با سطح

دسترسی سرویس SSH (که اغلب root است) روی سیستم هدف اجرا کند. این حمله به عنوان "ربودن

کانال قبل از احراز هویت " (pre-auth channel hijack) شناخته می‌شود.³¹

- تأثیر: این آسیب‌پذیری می‌تواند منجر به مصالحه کامل سیستم، نفوذ بدون برجای گذاشتن ردپای معمول (مانند لاگ‌های ناموفق ورود)، حرکت جانبی در شبکه، و حملات منع سرویس (DoS) شود.³¹

۴.۴. راه حل با مصالحه بین امنیت و عملکرد

ارائه یک راه‌حل که تعادل مناسبی بین امنیت و عملکرد برقرار کند، برای سیستم‌های نهفته با منابع محدود بسیار مهم است.

۱. وصله کردن: (Patching) راه‌حل اصلی و ارجح، اعمال وصله‌های امنیتی ارائه‌شده توسط نگه‌دارندگان Erlang/OTP است. نسخه‌های اصلاح‌شده (مانند OTP-26.2.5.11، OTP-27.3.3، OTP-28.2.2.20) این آسیب‌پذیری را برطرف می‌کنند.³¹ این اقدام پس از اعمال، تأثیر عملکردی حداقلی دارد و امنیت را به طور کامل بازیابی می‌کند.
۲. محدودسازی دسترسی شبکه: (Network Access Restriction) تا زمان اعمال وصله، می‌توان با استفاده از فایروال، دسترسی به پورت SSH سرور Erlang/OTP را فقط به آدرس‌های IP مورد اعتماد محدود کرد. این کار سطح حمله را بدون تأثیر بر عملکرد Erlang برای کاربران مجاز کاهش می‌دهد.³¹
۳. غیرفعال کردن سرویس SSH (اگر حیاتی نیست): اگر عملکرد SSH سرور Erlang/OTP برای سیستم حیاتی نیست، غیرفعال کردن موقت آن تا زمان اعمال وصله، آسیب‌پذیری را به طور کامل حذف می‌کند. تأثیر عملکردی آن، حذف بار پردازشی مربوط به سرویس SSH است.³¹
۴. جداسازی شبکه: (Network Segmentation) اطمینان حاصل کنید که گره Erlang فقط از طریق یک شبکه مدیریت امن یا VPN قابل دسترسی است. این یک لایه امنیتی اضافی ایجاد می‌کند. تأثیر عملکردی ممکن است به دلیل سربار VPN برای دسترسی‌های مجاز از راه دور، جزئی باشد.³¹
۵. سخت‌سازی عمومی سیستم و کاهش سطح حمله:
 - حذف سرویس‌ها و کتابخانه‌های غیرضروری: این کار باعث کاهش ردپای حافظه و تعداد آسیب‌پذیری‌های بالقوه می‌شود. تأثیر عملکردی آن اغلب مثبت است (سربار کمتر).³²
 - استفاده از Seccomp: محدود کردن فراخوان‌های سیستمی (syscalls) مجاز برای هر فرآیند با استفاده از Seccomp، سطح حمله را کاهش می‌دهد. این اقدام امنیتی، سربار عملکردی بالقوه حداقلی برای فیلتر کردن فراخوان‌های سیستمی دارد.³²
 - بوت امن: (Secure Boot) تضمین می‌کند که فقط نرم‌افزار مورد اعتماد در طول فرآیند بوت

بارگذاری می‌شود. این یک مزیت امنیتی با تأثیر حداقلی بر زمان بوت است.³³

- به‌روزرسانی‌های منظم و وصله‌گذاری زنده (Live Patching) استفاده از راه‌حل‌هایی مانند KernelCare IoT (از TuxCare) امکان اعمال وصله‌های امنیتی به توزیع‌های لینوکس نهفته مانند Yocto Project، Ubuntu Core و Raspberry Pi OS را بدون نیاز به راه‌اندازی مجدد سیستم و پنجره‌های نگهداری فراهم می‌کند. این یک مزیت امنیتی بزرگ است که از توقف سرویس جلوگیری می‌کند و تأثیر عملکردی خود وصله‌گذاری زنده معمولاً ناچیز است.³²

مصالحه امنیت و عملکرد: برای آسیب‌پذیری خاص CVE-2025-32433، وصله کردن راه‌حل ایده‌آل است. اگر وصله‌گذاری فوری امکان‌پذیر نباشد، محدودسازی دسترسی شبکه بهترین تعادل را ارائه می‌دهد. سخت‌سازی عمومی سیستم، امنیت کلی را با تأثیرات عملکردی اغلب مثبت یا خنثی بهبود می‌بخشد. بسیاری از آسیب‌پذیری‌های حیاتی در سیستم‌های لینوکس نهفته نه از نقص‌های ذاتی خود لینوکس، بلکه از پیکربندی‌های ناامن، اجزای قدیمی یا آسیب‌پذیری در برنامه‌های کاربردی فضای کاربر و سرویس‌های همراه با سیستم ناشی می‌شوند. این امر اهمیت یک رویکرد امنیتی جامع فراتر از صرفاً سخت‌سازی هسته را برجسته می‌کند. معاوضه "امنیت در برابر عملکرد" در لینوکس نهفته اغلب به اشتباه تعبیر می‌شود. در حالی که برخی اقدامات امنیتی سنگین مانند رمزنگاری کامل دیسک روی دستگاه‌های کم‌توان می‌تواند بر عملکرد تأثیر بگذارد، بسیاری از بهترین شیوه‌ها (مانند حذف سرویس‌های غیرضروری، تنظیمات پیش‌فرض امن، وصله‌گذاری) در واقع سطح حمله را کاهش داده و حتی می‌توانند با حذف سربار غیرضروری، عملکرد را بهبود بخشند. "هزینه" اصلی در اینجا اغلب افزایش نظم و انضباط در توسعه و نگهداری است، نه یک ضربه مستقیم به عملکرد.

۵) در توسعه سیستم های امبدد لینوکس بر روی برد هایی نظیر BeagleBone یا Raspberry Pi، مفهوم Device Tree Overlay چیست و به چه صورت در زمان اجرا به صورت پویا پرفرمانس های سخت افزاری را پیکربندی میکند؟ مثال واقعی شرح دهید.

در سیستم های لینوکس نهفته، به ویژه آن هایی که بر پایه معماری ARM یا سایر معماری های غیر x86 هستند، از "درخت دستگاه" (Device Tree) برای توصیف سخت افزار سیستم به هسته لینوکس استفاده می شود. "پوشش درخت دستگاه" (Device Tree Overlay - DTO) مکانیزمی قدرتمند برای اصلاح و سفارشی سازی این توصیفات سخت افزاری به صورت پویا است.

۵.۱. معرفی Device Tree و نیاز به Overlay

درخت دستگاه (DT) یک ساختار داده سلسله مراتبی است که سخت افزارهای موجود در یک سیستم را که قابلیت شناسایی خودکار ندارند (مانند کنترلرهای سریال، اترنت، یا حافظه فلش که مستقیماً روی یک System on Chip - SoC پیاده سازی شده اند) توصیف می کند.³⁴ این ساختار به هسته لینوکس اجازه می دهد تا با یک ایمپلیمنت دودویی یکسان روی پلتفرم های سخت افزاری مختلف با پیکربندی های متفاوت اجرا شود، زیرا اطلاعات خاص سخت افزار از کد هسته جدا شده و در DT ارائه می شود.³⁶ فایل منبع درخت دستگاه (DTS - Device Tree Source) به یک فایل دودویی کامپایل شده (DTB - Device Tree Blob) تبدیل می شود که توسط بوت لودر به هسته ارسال می گردد.

نیاز به پوشش های درخت دستگاه (DTO) زمانی مطرح می شود که بخواهیم درخت دستگاه پایه یک برد را برای پشتیبانی از سخت افزارهای جانبی (مانند capes در BeagleBone یا HATs در Raspberry Pi)، تغییر در مالتی پلکسینگ پین ها (pin multiplexing) یا اصلاح خصوصیات مرتبط با سیستم عامل (مانند خط فرمان هسته یا پارتیشن های فلش) سفارشی کنیم، بدون اینکه نیازی به تغییر و کامپایل مجدد کل DTB پایه یا خود هسته باشد.³⁴

۵.۲. Device Tree Overlay (DTO) چیست؟

یک DTO قطعه ای از یک درخت دستگاه است که برای اصلاح یک درخت دستگاه پایه موجود استفاده می شود.³⁴ این قطعات به صورت فایل های منبع dts نوشته شده و سپس به فایل های دودویی dtbo (Device Tree Blob Overlay) کامپایل می شوند.³⁴ این فایل های dtbo می توانند در زمان بوت (مثلاً توسط بوت لودر U-

Boot یا بوت‌لودر (Raspberry Pi) یا به طور بالقوه در زمان اجرای هسته به صورت زنده، روی DTB پایه اعمال شوند.³⁴

۵.۳. نحوه پیکربندی پویای پریفرال‌ها در زمان اجرا

DTOها امکان پیکربندی پویای سخت‌افزار را به دو روش اصلی فراهم می‌کنند:

- در زمان بوت:
 - U-Boot: بوت‌لودر U-Boot از دستوری مانند `fdt apply` پشتیبانی می‌کند که یک فایل `dtbo` بارگذاری شده در حافظه را با DTB پایه بارگذاری شده ادغام می‌کند. سپس DTB اصلاح‌شده به هسته ارسال می‌شود.³⁴
 - Raspberry Pi: در Raspberry Pi، دستور `dtoverlay` در فایل `config.txt` (واقع در پارتیشن بوت) به بوت‌لودر دستور می‌دهد که فایل `dtbo` مشخص شده را از پوشه `/boot/overlays/` بارگذاری کرده و روی DTB پایه اعمال کند.³⁷
- در زمان اجرای کرنل (Live Kernel Application):
 - هسته لینوکس دارای یک API داخلی است که شامل توابعی مانند `of_overlay_fdt_apply` (برای ایجاد و اعمال یک مجموعه تغییرات DTO و `of_overlay_remove` برای حذف آن است).³⁹ این قابلیت امکان ثبت (`register`) و لغو ثبت (`deregister`) پویای دستگاه‌ها را فراهم می‌کند.
 - اگرچه این قابلیت قدرتمند است، استفاده از آن برای پیکربندی کامل پریفرال‌ها به صورت زنده کمتر از اعمال در زمان بوت رایج است. در گذشته اشاره شده که بارگذاری زنده DTOها برای `capex` پیچیده هنوز به بلوغ کامل نرسیده است.³⁴ با این حال، برای تنظیمات پویای جزئی‌تر می‌تواند مفید باشد.

۵.۴. مثال واقعی (BeagleBone با Relay Cape)

در ادامه مثالی از استفاده DTO روی برد BeagleBone Black برای فعال‌سازی و کنترل یک Relay Cape ارائه می‌شود.

- کد (`.dts`) DTO برای Relay Cape:

هدف این DTO، پیکربندی پین‌های GPIO متصل به رله‌ها و تعریف آن‌ها به عنوان دستگاه‌های LED در لینوکس برای کنترل آسان است.

- نحوه هدف قرار دادن گره‌های درخت دستگاه پایه:
 - `&{/chosen}`: گره `chosen/` در DTB پایه را هدف قرار می‌دهد تا اطلاعات مربوط به `overlay` بارگذاری شده را اضافه کند.
 - `&ocp`: گره `ocp (On-Chip Peripherals)` را هدف قرار می‌دهد تا تنظیمات پیش‌فرض پین‌ماکس برای پین‌هایی که توسط `Relay Cape` استفاده می‌شوند را غیرفعال کند و از تداخل جلوگیری نماید.
 - `&am33xx_pinmux`: کنترلر پین‌ماکسینگ (`AM33xx`) SoC را هدف قرار می‌دهد تا پین‌های مورد نظر را برای عملکرد GPIO با مقاومت `pull-down` و حالت مالتی‌پلکسینگ ۷ (که معمولاً برای GPIO است) پیکربندی کند.
 - `&{/}`: ریشه درخت دستگاه را هدف قرار می‌دهد تا یک گره جدید `leds` اضافه کند. این یک روش متداول برای کنترل GPIOهای ساده از طریق رابط استاندارد `gpio-leds` در لینوکس است که دستگاه‌ها را در مسیر `/sys/class/leds/` در دسترس قرار می‌دهد.
- مراحل کامپایل و اعمال/بارگذاری DTO (مثال برای BeagleBone):
 ۱. کامپایل DTO:
 - اطمینان از نصب بودن کامپایلر درخت دستگاه (`dtc`).
 - اگر DTO شامل دستورات پیش‌پردازنده C (مانند `#include` یا ماکروهایی مثل `__TIMESTAMP__`) باشد، باید از طریق `Makefile` هسته لینوکس کامپایل شود.³⁴ این کار با تنظیم متغیرهای محیطی `ARCH` و `CROSS_COMPILE`، اجرای `make omap2plus_defconfig` (یا `defconfig` مربوط به برد)، فعال کردن گزینه‌های `CONFIG_LEDS_GPIO=y`، `CONFIG_OF_OVERLAY=y` و `CONFIG_LEDS_CLASS=y` از طریق `make menuconfig` و سپس اجرای `make dtbs` انجام می‌شود. فایل `dtbo` (مثلاً `BBORG_RELAY-00A2.dtbo`) در پوشه `/arch/arm/boot/dts/overlays` (بسته به نسخه کرنل) ایجاد می‌شود.
 ۲. کپی کردن فایل‌ها به کارت SD: فایل `dtbo` کامپایل شده، DTB پایه برد (مثلاً `am335x-boneblack.dtbo`) و ایمیج هسته (مثلاً `zImage`) باید در پارتیشن بوت کارت SD (معمولاً با فرمت FAT) کپی شوند.
 ۳. اعمال DTO در U-Boot: در خط فرمان U-Boot، دستورات زیر (یا مشابه آن‌ها، بسته به پیکربندی U-Boot و محل فایل‌ها) اجرا می‌شوند:³⁴

۴. بررسی در لینوکس: پس از بوت شدن سیستم عامل لینوکس، می توان با بررسی محتوای `/proc/device-tree/chosen/overlays/` از اعمال شدن overlay اطمینان حاصل کرد. برای مثال Relay Cape، باید نام `BBORG_RELAY-00A2.kernel` در این مسیر دیده شود. همچنین، رله ها باید به عنوان LED در مسیر `/sys/class/leds/` (مثلاً `/sys/class/leds/relay/`) قابل دسترس و کنترل باشند.³⁴

- مثال برای Raspberry Pi (فعال سازی SPI):
در Raspberry Pi، فرآیند اعمال DTO معمولاً از طریق فایل `config.txt` در پارتیشن بوت انجام می شود.
 - کد (`.dts`) DTO برای فعال سازی SPI0 :
 - کامپایل: `dtc -@ -I dts -O dtb -o spi-overlay.dtbo spi-overlay.dts`
 - اعمال در `config.txt`: فایل `spi-overlay.dtbo` را به پوشه `/boot/overlays/` در کارت SD کپی کنید. سپس خط زیر را به فایل `boot/config.txt/` اضافه کنید: `dtoverlay=spi-overlay` (توجه: برای بسیاری از پرفرمانس های رایج، Raspberry Pi Foundation پوشش های از پیش تعریف شده ای ارائه می دهد، مثلاً `dtoverlay=spi0-cs` یا `dtoverlay=spi-bcm2835` که نیازی به کامپایل دستی ندارند).
 - نتیجه: پس از راه اندازی مجدد، رابط SPI فعال شده و دستگاه های مربوطه (مانند `/dev/spidevX.Y/`) در سیستم عامل ظاهر می شوند.

۵.۵. مزایای استفاده از DTO

- استفاده از DTO ها مزایای متعددی در توسعه سیستم های لینوکس نهفته به همراه دارد:
- ماژولار بودن و قابلیت استفاده مجدد: DTO ها امکان تعریف پیکربندی های سخت افزاری به صورت ماژولار را فراهم می کنند که می توانند به طور مستقل توسعه یافته و روی بردهای مختلف (با DTB پایه سازگار) استفاده شوند.³⁶
 - ساده سازی فرآیند آماده سازی برد (Board Bring-up) و سفارشی سازی: توسعه دهندگان می توانند به سرعت سخت افزارهای جدید را با نوشتن یا اعمال DTO های موجود فعال کنند، بدون نیاز به درگیر شدن با جزئیات پیچیده کد هسته.³⁶
 - عدم نیاز به کامپایل مجدد هسته یا DTB کامل: برای تغییرات رایج سخت افزاری (مانند فعال کردن یک رابط یا اتصال یک ماژول جدید)، نیازی به کامپایل مجدد کل هسته یا DTB پایه نیست که باعث

صرفه جویی در زمان توسعه می شود.³⁵

- تسهیل پشتیبانی از چندین نسخه برد با یک ایمپلیمنت هسته: تولیدکنندگان می توانند یک ایمپلیمنت هسته استاندارد ارائه دهند و کاربران با اعمال DTOهای مناسب، آن را برای نسخه های مختلف برد یا پیکربندی های سخت افزاری خاص خود تنظیم کنند.³⁶
- ترویج همکاری در جامعه توسعه دهندگان: DTOها به راحتی قابل اشتراک گذاری هستند، که امکان استفاده از تجربیات دیگران برای فعال سازی پریفرال ها یا ماژول های رایج را فراهم می کند.³⁶

Device Tree Overlays یک لایه انتزاعی مهم بین تعریف سخت افزار پایه برد و پیکربندی های خاص برنامه یا افزونه ها ایجاد می کنند. این امر چرخه عمر محصول و سفارشی سازی را به طور قابل توجهی بهبود می بخشد. تولیدکنندگان برد می توانند یک DTB پایه پایدار ارائه دهند و توسعه دهندگان برنامه های کاربردی یا سخت افزارهای جانبی، DTOهای خود را به طور مستقل توسعه دهند. این ماژولار بودن، مدیریت پیکربندی های پیچیده و به روزرسانی های نرم افزاری را ساده تر می کند.

قابلیت بارگذاری DTO در زمان اجرای کرنل، پتانسیل ایجاد سیستم های "خودپیکربند" (self-configuring) یا پشتیبانی بهتر از "پلاگین و پخش داغ" (hot-plug and play) را برای سخت افزارهایی که به طور خودکار قابل شناسایی نیستند، در لینوکس نهفته فراهم می کند. با این حال، این قابلیت چالش هایی را نیز در زمینه پایداری و مدیریت منابع به همراه دارد. تغییرات دینامیک در درخت دستگاه می تواند پیچیده باشد و درایورها باید بتوانند با اضافه و حذف شدن ناگهانی دستگاه ها سازگار باشند. اطمینان از آزادسازی صحیح تمام منابع پس از حذف یک DTO و جلوگیری از باقی ماندن اشاره گرهای نامعتبر در کرنل، یک چالش مهم است³⁹ که نیازمند طراحی دقیق درایورها و استفاده محتاطانه از DTOهای زنده است.

سینتکس و ابزارهای Device Tree (شامل Overlays) خود یک "زبان توصیف سخت افزار" استاندارد شده را تشکیل می دهند که همکاری بین تولیدکنندگان SoC، تولیدکنندگان برد، و توسعه دهندگان نرم افزار را تسهیل می کند. این استانداردسازی قابلیت استفاده مجدد از کد و دانش را افزایش می دهد.³⁶ با این حال، یادگیری سینتکس DTS، مفاهیم گره ها، خصوصیات، برچسب ها، phandleها و نحوه صحیح نوشتن یک DTO که با DTB پایه به درستی ادغام شود، برای تازه واردان می تواند چالش برانگیز باشد.

بخش عملی

الف) همانطور که در کلاس درس هم اشاره شده است، اکثر کد کرنل لینوکس به زبان C نوشته شده است ولی برخی بخش های آن به زبان اسمبلی است. تحقیق کنید که اولاً دقیقاً آن بخش هایی که به زبان اسمبلی نوشته شده اند چه هستند و ثانیاً اینکه علت هر یک از آنها به چه دلیلی است. در این باره تحقیق کنید.

۱.۱: زبان های برنامه نویسی در توسعه کرنل

کرنل لینوکس عمدتاً با استفاده از زبان برنامه نویسی C توسعه یافته است.⁴¹ انتخاب زبان C به دلایل متعددی صورت گرفته است که از جمله مهم ترین آنها می توان به قابلیت حمل بالا (portability) بین معماری های مختلف سخت افزاری، کارایی مناسب و توانایی دسترسی سطح پایین به حافظه و سخت افزار اشاره کرد. GCC (مجموعه کامپایلرهای گنو) کامپایلر اصلی مورد استفاده برای ساخت کرنل است و از نسخه ای خاص از زبان C با افزونه های متعدد پشتیبانی می کند.⁴¹

با وجود تسلط زبان C، بخش های خاصی از کرنل به زبان اسمبلی نوشته شده اند. زبان اسمبلی، به عنوان یک زبان سطح پایین که مستقیماً با دستورالعمل های پردازنده سروکار دارد، برای انجام وظایفی که فراتر از توانایی های استاندارد زبان C هستند یا نیازمند کنترل بسیار دقیق بر روی سخت افزار می باشند، یک ابزار ضروری محسوب می شود.

در سال های اخیر، جامعه توسعه دهندگان کرنل لینوکس شاهد معرفی و پشتیبانی اولیه از زبان برنامه نویسی Rust نیز بوده است. پشتیبانی از Rust برای اولین بار در نسخه ۶.۱ کرنل لینوکس (منتشر شده در دسامبر ۲۰۲۲) اضافه شد و در نسخه های بعدی مانند ۶.۲ و ۶.۳ بهبود یافت.⁴¹ هدف اصلی از معرفی Rust، افزایش ایمنی حافظه (memory safety) و کاهش خطاهای مرتبط با آن است که یکی از چالش های همیشگی در توسعه با زبان C، به ویژه در محیط پیچیده ای مانند کرنل، بوده است. این تلاش برای استفاده از زبان های جدیدتر با قابلیت های ایمنی بیشتر، خود نشان دهنده تکامل مداوم در رویکردهای توسعه کرنل است و به طور غیرمستقیم، دلایل استفاده از اسمبلی را نیز روشن تر می کند: زبان C، با تمام مزایایش، در برخی حوزه ها (مانند تعامل بسیار نزدیک با سخت افزار یا تضمین ایمنی حافظه در تمام شرایط) دارای محدودیت هایی است که

توسعه‌دهندگان را به سمت استفاده از اسمبلی (برای کنترل سخت‌افزار) یا زبان‌های مدرن‌تر مانند Rust (برای افزایش ایمنی) سوق می‌دهد.

۱.۲: دلایل اصلی استفاده از زبان اسمبلی در کرنل

استفاده از زبان اسمبلی در کرنل لینوکس یک تصمیم سبک‌سرا نه نیست، بلکه ضرورتی است که از محدودیت‌های زبان‌های سطح بالاتر مانند C در تعامل با جنبه‌های خاص سخت‌افزار و سیستم نشأت می‌گیرد. دلایل اصلی این امر به شرح زیر است:

- تعامل مستقیم با سخت‌افزار و دسترسی به ویژگی‌های خاص پردازنده: بسیاری از دستورالعمل‌های پردازنده که برای عملکرد صحیح و کارآمد یک سیستم عامل ضروری هستند، معادل مستقیمی در زبان C استاندارد ندارند. زبان اسمبلی به توسعه‌دهندگان اجازه می‌دهد تا مستقیماً از این دستورالعمل‌ها استفاده کنند. مثال‌هایی از این موارد عبارتند از:
 - فعال‌سازی حالت‌های خاص پردازنده: مانند ورود به حالت ۱۶ بیتی در پردازنده‌های Intel برای اجرای کدهای اولیه بوت یا firmware.⁴²
 - دسترسی به رجیسترهای کنترلی CPU: رجیسترهایی که وضعیت و عملکرد پردازنده را کنترل می‌کنند.
 - دستورات اتمی (Atomic operations): برای انجام عملیاتی که باید بدون وقفه و به صورت یکپارچه انجام شوند، حیاتی برای همگام‌سازی در سیستم‌های چندپردازنده‌ای.
 - دستورات قفل کردن باس (Bus locking instructions): برای تضمین دسترسی انحصاری به حافظه مشترک.
 - خواندن شمارنده برچسب زمانی (Timestamp counter register reading): برای اندازه‌گیری دقیق زمان.
 - عملیات ورودی/خروجی (I/O operations): دستورات مستقیم برای ارتباط با پورت‌های ورودی/خروجی.
 - دستورات خاص مانند AES برای رمزنگاری سخت‌افزاری.⁴³
- بوت‌استرپینگ و مقداردهی اولیه سیستم: (Bootstrapping and Early System Initialization) هنگامی که یک کامپیوتر روشن می‌شود، در وضعیتی قرار ندارد که کد نوشته شده به زبان C بتواند فوراً اجرا شود. پردازنده در یک حالت اولیه قرار دارد و بسیاری از زیرساخت‌های مورد نیاز C، مانند پشته

(stack) یا مدیریت حافظه مجازی (paging)، هنوز پیکربندی نشده‌اند. زبان اسمبلی برای انجام این وظایف اولیه و حیاتی استفاده می‌شود:

- مقداردهی اولیه حافظه و کنترلرهای حافظه.
- تنظیم ثبات‌های CPU در وضعیت مناسب برای اجرای کدهای بعدی.
- ایجاد و تنظیم پشته اولیه.
- فعال‌سازی ویژگی‌های پایه‌ای پردازنده مانند حالت محافظت‌شده (protected mode) و صفحه‌بندی (paging).⁴² این کد اسمبلی، سیستم را به مرحله‌ای می‌رساند که بتواند کنترل را به بخش‌های نوشته شده به زبان C (مانند تابع start_kernel) منتقل کند.
- مدیریت وقفه‌ها، استثناها و تله‌ها: (Interrupt, Exception, and Trap Handling)
 - نقاط ورود و خروج وقفه‌ها (interrupts)، استثناها (exceptions) و تله‌های نرم‌افزاری (traps) نیازمند تعامل بسیار دقیق با معماری پردازنده هستند. این عملیات شامل موارد زیر است:
 - ذخیره کامل و صحیح وضعیت فعلی پردازنده (تمام رجیسترهای مرتبط).
 - انتقال کنترل به روال گرداننده وقفه (interrupt handler).
 - پس از اتمام کار گرداننده، بازیابی دقیق وضعیت پردازنده.
 - فعال‌سازی مجدد وقفه‌ها (در صورت لزوم).
 - بازگشت صحیح به کدی که اجرای آن متوقف شده بود، چه در حالت کرنل و چه در حالت کاربر (مثلاً با استفاده از دستوراتی مانند iret در معماری x86)⁴² برخی از استثنائات ممکن است به طور کامل در کد اسمبلی مدیریت شوند تا از سربار اضافی جلوگیری شود.⁴²
- واسطه فراخوانی سیستمی: (System Call Interface)
 - انتقال کنترل از برنامه‌های فضای کاربر به کرنل برای درخواست خدمات سیستم‌عامل (فراخوانی‌های سیستمی) یک عملیات حساس و حیاتی است. اسمبلی در این بخش برای موارد زیر کاربرد دارد:
 - انتخاب توالی بهینه و امن دستورالعمل‌ها برای ورود به کرنل، بسته به نوع CPU (مثلاً در معماری x86، استفاده از دستورات جدیدتر و سریع‌تر مانند sysenter یا syscall به جای دستور قدیمی int 0x80 در صورت پشتیبانی توسط پردازنده).⁴²
 - پیاده‌سازی برخی فراخوانی‌های سیستمی خاص که می‌توانند بدون نیاز به تعویض زمینه کامل (full context switch) انجام شوند، که به بهبود کارایی کمک می‌کند (مثلاً برخی جنبه‌های gettimeofday).⁴²

- بهینه‌سازی کارایی: (Performance Optimization - Historically More Prevalent)
در گذشته، یکی از دلایل استفاده از اسمبلی، دستیابی به حداکثر کارایی در بخش‌های بسیار حساس و پرتکرار کد بود. با این حال، کامپایلرهای مدرن زبان C به شدت پیشرفت کرده‌اند و اغلب کدی تولید می‌کنند که از نظر سرعت با کد اسمبلی دست‌نویس برابری می‌کند یا حتی از آن بهتر است. بنابراین، امروزه استفاده از اسمبلی صرفاً برای بهینه‌سازی کارایی بسیار کمتر رایج شده است، مگر در موارد بسیار خاص و محدود که کامپایلر قادر به تولید کد بهینه نباشد یا نیاز به استفاده از ویژگی‌های خاص پردازنده برای افزایش سرعت وجود داشته باشد. اولویت اصلی در کرنل لینوکس، قابلیت حمل و خوانایی کد است.
- نیاز به نشانه‌گذاری دقیق کد و داده: (Precise Marking of Code and Data)
اسمبلرهای استاندارد معمولاً کاربران را مجبور به نشانه‌گذاری دقیق بخش‌های مختلف کد، داده‌ها، یا حتی طول آن‌ها نمی‌کنند. اما در توسعه کرنل، به ویژه برای اهداف اشکال‌زدایی (debugging) و عملکرد صحیح ابزارهایی مانند objtool، این نشانه‌گذاری دقیق حیاتی است. زبان اسمبلی، به همراه ماکروهای خاص تعریف شده در کرنل، این امکان را فراهم می‌کند تا توابع و داده‌ها به طور صریح به عنوان کد یا داده، و با طول مشخص، تعریف شوند. این امر به تولید فایل‌های آبجکت (object files) خوش‌ساختار با اطلاعات دقیق در مورد اندازه و نوع نمادها کمک می‌کند، که برای تحلیل‌های پس از کامپایل و اشکال‌زدایی ضروری است.

تصمیم به استفاده از زبان اسمبلی در کرنل لینوکس با آگاهی کامل از معایب آن، به خصوص کاهش قابلیت حمل (portability) و افزایش پیچیدگی نگهداری (maintainability) کد، اتخاذ می‌شود.⁴³ هر بخش نوشته شده به اسمبلی باید برای هر معماری سخت‌افزاری جدیدی که کرنل قصد پشتیبانی از آن را دارد، بازنویسی یا تطبیق داده شود. این امر نشان می‌دهد که اسمبلی تنها زمانی به کار گرفته می‌شود که یک "ضرورت اجتناب‌ناپذیر" وجود داشته باشد؛ یعنی برای انجام وظایفی که زبان C قادر به انجام آن‌ها نیست یا نمی‌تواند با سطح کنترل و دقت مورد نیاز در تعامل با سخت‌افزار انجام دهد. موفقیت کرنل لینوکس در اجرا بر روی طیف وسیعی از معماری‌های سخت‌افزاری⁴³، گواهی بر تلاش موفقیت‌آمیز برای به حداقل رساندن و محصور کردن این بخش‌های وابسته به اسمبلی است.

۱.۳: محل و نمونه‌های کد اسمبلی در ساختار کرنل

کد اسمبلی در کرنل لینوکس به صورت پراکنده و بدون ساختار استفاده نمی‌شود. بلکه، به طور عمده در دایرکتوری‌های مختص معماری سخت‌افزار، یعنی زیرشاخه‌های دایرکتوری arch/ در درخت سورس کرنل، یافت

می‌شود.⁴² این سازماندهی به جداسازی بخش‌های غیرقابل حمل و وابسته به پلتفرم از بدنه اصلی و قابل حمل کرنل که به زبان C نوشته شده است، کمک شایانی می‌کند. در ادامه به بررسی محل و نمونه‌هایی از این کدها می‌پردازیم:

- فرایند بوت (Boot Process) - مثال برای معماری x86 (i386):

فرایند راه‌اندازی اولیه سیستم (بوت) یکی از اصلی‌ترین بخش‌هایی است که به شدت به کد اسمبلی وابسته است. برای نمونه، در معماری x86 (مشخصاً پردازنده‌های i386)، دنباله‌ای از فایل‌های اسمبلی نقش کلیدی ایفا می‌کنند:

- ورود اولیه به کرنل از طریق یک فایل اسمبلی به نام `head.S` صورت می‌گیرد.⁴²
- فایل `arch/i386/boot/head.S`: شامل روتین اسمبلی `start` است که وظیفه انجام تنظیمات بسیار پایه‌ای سخت‌افزار را بر عهده دارد و سپس کنترل را به روتین `۳۲_startup` منتقل می‌کند.⁴⁵
- فایل `arch/i386/boot/compressed/head.S`: این فایل حاوی روتین `۳۲_startup` دیگری است که پس از اجرای `start` فراخوانی می‌شود. وظایف آن شامل تنظیم محیط اولیه اجرایی مانند پشته (`stack`)، پاک‌سازی بخش `BSS (Block Started by Symbol)` از حافظه (که برای متغیرهای سراسری مقداره‌ی نشده استفاده می‌شود) و سپس فراخوانی تابع C به نام `decompress_kernel` برای خارج‌سازی ایمج فشرده کرنل از حالت فشرده است.⁴⁵
- فایل `arch/i386/kernel/head.S`: پس از اینکه کرنل از حالت فشرده خارج و در حافظه بارگذاری شد، یک روتین `32_startup` دیگر، که در این فایل قرار دارد، اجرا می‌شود. این روتین مسئول برقراری مدیریت حافظه اساسی، شامل جداول صفحه‌بندی (`paging tables`) و فعال‌سازی صفحه‌بندی است. همچنین، نوع پردازنده و قابلیت‌های اضافی آن (مانند واحد ممیز شناور) را تشخیص می‌دهد. در نهایت، این کد اسمبلی کنترل را به تابع C به نام `start_kernel` (که در فایل `init/main.c` قرار دارد و نقطه شروع مستقل از معماری کرنل محسوب می‌شود) واگذار می‌کند.⁴⁴

- اسمبلی درون‌خطی (Inline Assembly):

کامپایلر GCC این قابلیت را فراهم می‌کند که قطعات کوچکی از کد اسمبلی مستقیماً در درون توابع C گنجانده شوند. این ویژگی به عنوان "اسمبلی درون‌خطی" شناخته می‌شود. کرنل لینوکس از این قابلیت با استفاده از سینتکس "AT&T-style" که توسط GCC پشتیبانی می‌شود، بهره می‌برد. اسمبلی درون‌خطی عمدتاً برای دسترسی به ویژگی‌های خاص CPU یا رجیسترهایی که مستقیماً از طریق زبان C قابل دسترسی نیستند، استفاده می‌شود. این قطعات کد اسمبلی معمولاً کوتاه و مختصر هستند و برای انجام

یک وظیفه بسیار خاص طراحی شده‌اند.

- ماکروهای کرنل برای اسمبلی: (Kernel Macros for Assembly)
به منظور افزایش خوانایی، قابلیت نگهداری و استانداردسازی استفاده از اسمبلی، کرنل لینوکس مجموعه‌ای از ماکروهای پیش‌پردازنده را تعریف کرده است. این ماکروها، که معمولاً با پیشوند SYM_ شروع می‌شوند (مانند SYM_FUNC_START, SYM_FUNC_END, SYM_CODE_START, SYM_CODE_END, SYM_DATA_START و غیره)، برای نشانه‌گذاری و سازماندهی توابع و داده‌های تعریف شده در زبان اسمبلی به کار می‌روند
استفاده صحیح از این ماکروها به اسمبلر کمک می‌کند تا فایل‌های آبجکت خوش‌ساختار با اطلاعات دقیق در مورد اندازه و نوع نمادها (symbols) تولید کند. این اطلاعات برای فرآیند اشکال‌زدایی (debugging) و همچنین برای ابزارهای تحلیل کد مانند objtool بسیار حیاتی است. objtool می‌تواند با استفاده از این نشانه‌گذاری‌ها، بررسی‌های صحت‌سنجی انجام دهد (مثلاً اطمینان از تنظیم صحیح اشاره‌گر فریم) و فراداده‌های لازم برای مکانیزم‌های unwinder (که برای تولید ردپای پشت‌پشته یا stack trace استفاده می‌شوند) را تولید کند. ماکروهای SYM_FUNC_* برای توابعی استفاده می‌شوند که از قراردادهای فراخوانی استاندارد C پیروی می‌کنند، در حالی که ماکروهای SYM_CODE_* برای توابع خاص با قراردادهای غیراستاندارد (مانند گرداننده‌های وقفه یا trampolines) به کار می‌روند.
- کد مختص معماری برای عملیات دیگر:

علاوه بر بوت، بخش‌های دیگری نیز وجود دارند که کد اسمبلی مختص معماری در آن‌ها یافت می‌شود. این موارد شامل پیاده‌سازی‌های بهینه شده برای معماری خاص از توابع رایج مانند memcpy (کپی حافظه) و سایر عملیات رشته‌ای است. ۹. همچنین، کد اسمبلی در مدیریت اجزای سخت‌افزاری خاص معماری مانند کنترلرهای وقفه، کنترلرهای SMP (Symmetric MultiProcessing)، کنترلرهای باس (BUS controllers)، تنظیمات مربوط به استثناها و وقفه‌ها، و جنبه‌های سطح پایین مدیریت حافظه مجازی نقش دارد.

استفاده ساختاریافته از زبان اسمبلی، به ویژه از طریق به‌کارگیری ماکروهای استاندارد و محدود کردن بخش عمده آن به دایرکتوری‌های arch، یک استراتژی آگاهانه در توسعه کرنل لینوکس است. این رویکرد به مدیریت پیچیدگی ذاتی کد اسمبلی کمک کرده و تأثیر منفی آن بر قابلیت حمل و نگهداری کل کدبیس را به حداقل می‌رساند. این نشان می‌دهد که استفاده از اسمبلی در کرنل، یک امر تصادفی و پراکنده نیست، بلکه بخشی

مهندسی شده و کنترل شده از فرآیند توسعه است. این ساختار دقیق به بدنه اصلی کرنل اجازه می‌دهد تا عمده‌تاً به زبان C قابل حمل باقی بماند، در حالی که "ضرورت‌های اجتناب‌ناپذیر" استفاده از اسمبلی به صورت محصور و مدیریت شده باقی می‌مانند. این رویکرد ساختاریافته، یکی از دلایل کلیدی موفقیت کرنل لینوکس در پشتیبانی از طیف گسترده‌ای از معماری‌های سخت‌افزاری است.

۱.۴: موازنه بین قابلیت حمل، کنترل و کارایی

یکی از اصول بنیادین در طراحی و توسعه کرنل لینوکس، تأکید بر قابلیت حمل (portability) آن بین معماری‌های سخت‌افزاری متنوع است.⁴³ این اصل، دلیل اصلی انتخاب زبان C به عنوان زبان غالب برای توسعه کرنل بوده است، زیرا C به طور گسترده‌ای توسط کامپایلرها برای معماری‌های مختلف پشتیبانی می‌شود و امکان نوشتن کدی را فراهم می‌کند که با حداقل تغییرات یا بدون تغییر، بر روی پلتفرم‌های گوناگون قابل کامپایل و اجرا باشد.

در این چارچوب، استفاده از زبان اسمبلی با دقت و وسواس زیادی صورت می‌گیرد. اسمبلی تنها در بخش‌هایی از کرنل به کار گرفته می‌شود که "کاملاً ضروری" تشخیص داده شود. این ضرورت معمولاً زمانی پیش می‌آید که:

۱. کامپایلر C قادر به تولید دستورالعمل‌های خاص پردازنده که برای یک عملکرد خاص مورد نیاز هستند، نباشد.
۲. نیاز به کنترل بسیار دقیق و سطح پایین بر روی سخت‌افزار وجود داشته باشد، به ویژه در مراحل اولیه راه‌اندازی سیستم (بوت) که هنوز محیط اجرایی C به طور کامل آماده نشده است.

استدلال مبتنی بر "کارایی" برای استفاده از اسمبلی در کرنل، امروزه تا حد زیادی اهمیت تاریخی خود را از دست داده است. همانطور که پیشتر اشاره شد، کامپایلرهای مدرن C بسیار پیشرفته شده‌اند و اغلب قادرند کدی تولید کنند که از نظر سرعت با کد اسمبلی دست‌نویس رقابت کند یا حتی از آن پیشی بگیرد.⁴³ بنابراین، دلایل اصلی استفاده از اسمبلی در کرنل‌های مدرن، بیشتر حول محور ضرورت (برای انجام کارهایی که C نمی‌تواند، مانند راه‌اندازی اولیه سیستم یا استفاده از دستورات خاص پردازنده) و کنترل (برای تعامل دقیق با سخت‌افزار) می‌چرخد، نه صرفاً افزایش عمومی سرعت نسبت به کد C.⁴² در واقع، برای بخش‌های کوچکی مانند توابع درون‌خطی برای فعال/غیرفعال کردن وقفه‌ها، هدف اصلی کسب کارایی نیست، بلکه انجام عملیاتی است که C مستقیماً قادر به انجام آن نیست.⁴³

با این حال، این کنترل و ضرورت با هزینه‌ای همراه است: هزینه نگهداری. کدهای نوشته شده به زبان اسمبلی ذاتاً وابسته به معماری خاص پردازنده هستند. این بدان معناست که برای هر معماری جدیدی که کرنل لینوکس قصد پشتیبانی از آن را دارد، این بخش‌های اسمبلی باید از نو نوشته یا به دقت تطبیق داده شوند.⁴³ این فرآیند نه تنها زمان‌بر است، بلکه پتانسیل ایجاد خطا را نیز افزایش می‌دهد. به همین دلیل، جامعه توسعه‌دهندگان کرنل همواره در تلاش است تا میزان استفاده از اسمبلی را به حداقل ممکن برساند و آن را در بخش‌های کاملاً ایزوله و مشخص (عمدتاً در دایرکتوری‌های arch/) محدود کند. این موازنه دقیق بین نیاز به کنترل سطح پایین، حفظ قابلیت حمل و مدیریت هزینه‌های نگهداری، یکی از چالش‌های دائمی در مهندسی کرنل لینوکس است.

حوزه/عملکرد کرنل	دلیل اصلی استفاده از اسمبلی	مثال (ها) (فایل/تابع/ماکرو نمونه)
بوت اولیه سیستم (Early System) (Boot)	مقداردهی سخت‌افزار قبل از آمادگی کامل محیط اجرایی C	arch/x86/boot/head.S start, startup_32), arch/x86/kernel/head.S (startup_32) ⁴⁵
دسترسی به دستورات خاص CPU (CPU- Specific Instructions)	اجرای دستوراتی که معادل مستقیمی در زبان C ندارند.	اسمبلی درون‌خطی برای دستورات مانند cli/sti (کنترل وقفه‌ها، I/O)in/out پورت) ⁴²
مدیریت وقفه‌ها و استثناها (Interrupt/Exception Handling)	نیاز به ذخیره و بازیابی دقیق وضعیت پردازنده و کنترل جریان اجرا.	کد ورودی/خروجی وقفه، معمولاً در فایل‌هایی مانند arch/[arch]/kernel/entry.S ⁴²
رابط فراخوانی سیستمی (System Call) (Interface)	انتقال کنترل بهینه و امن بین فضای کاربر و فضای کرنل.	کد ورودی به فراخوانی‌های سیستمی (مثلاً مسیر sysenter یا syscall در x86) ²⁴
مقداردهی اولیه حافظه و پشته (Memory/Stack Initialization)	تنظیم ساختارهای اولیه حافظه و پشته قبل از اجرای کد C.	بخش‌هایی از کد بوت در arch/[arch]/boot/ ⁴²
نشانه‌گذاری کد/داده اسمبلی (Assembly) (Code/Data Annotation)	ایجاد آبجکت‌فایل‌های دقیق برای ابزارها، اشکال‌زدایی و unwinding.	ماکروهای SYM_FUNC_START, SYM_CODE_START, SYM_DATA_START و نظایر آن‌ها ⁴⁶

این جدول به صورت خلاصه نشان می‌دهد که زبان اسمبلی، اگرچه بخش کوچکی از کل کدبیس کرنل را تشکیل می‌دهد، اما نقش حیاتی و غیرقابل جایگزینی در عملکرد صحیح و پایه‌ای سیستم‌عامل لینوکس ایفا می‌کند. استفاده از آن همواره با در نظر گرفتن موازنه بین کنترل، ضرورت و قابلیت حمل صورت می‌پذیرد.

ب) تحقیق کنید که اخیراً چه اشکالات یا باک‌هایی در سیستم عامل لینوکس پیدا شده است. به حداقل ۵ مورد از آنها با ذکر منبع (یا کد مربوطه) اشاره کنید و دقیق توضیح بدهید و زمان تشخیص و رفع آنها را نیز بنویسید.

۲.۱: مقدمه‌ای بر آسیب‌پذیری‌های کرنل و شناسه‌های CVE

کرنل لینوکس به عنوان بخش مرکزی و هسته اصلی سیستم‌عامل، مسئولیت مدیریت تمامی منابع سیستم و اجرای دستورات با بالاترین سطح دسترسی (privileged mode) را بر عهده دارد. به همین دلیل، امنیت کرنل از اهمیت فوق‌العاده‌ای برخوردار است و هرگونه آسیب‌پذیری در آن می‌تواند منجر به عواقب جدی، از جمله از کار افتادن سیستم، نشت اطلاعات حساس، یا حتی کنترل کامل سیستم توسط مهاجمان شود.

با توجه به حجم عظیم کد (میلیون‌ها خط) و پیچیدگی ذاتی کرنل، و همچنین تعداد زیاد توسعه‌دهندگانی که به طور مداوم بر روی آن کار می‌کنند، کشف باگ‌ها و آسیب‌پذیری‌های امنیتی امری اجتناب‌ناپذیر است. برای استانداردسازی فرآیند شناسایی، تعریف و طبقه‌بندی این آسیب‌پذیری‌ها، از سیستم CVE (Common Vulnerabilities and Exposures) استفاده می‌شود. هر آسیب‌پذیری عمومی شناخته‌شده یک شناسه CVE منحصر به فرد دریافت می‌کند که به متخصصان امنیت و مدیران سیستم اجازه می‌دهد تا به طور دقیق به یک مشکل خاص ارجاع دهند.

فرآیند گزارش، تحقیق و رفع آسیب‌پذیری‌ها در جامعه متن‌باز و به ویژه در پروژه کرنل لینوکس، یک فرآیند مشارکتی و شفاف است. توسعه‌دهندگان و محققان امنیتی، آسیب‌پذیری‌های کشف‌شده را معمولاً از طریق لیست‌های پستی اختصاصی (مانند لیست پستی امنیتی کرنل) یا به نگهدارندگان (maintainers) زیرسیستم‌های مربوطه گزارش می‌دهند.⁴¹ پس از تأیید آسیب‌پذیری، یک پچ (patch) یا اصلاحیه تهیه شده و پس از بررسی و آزمایش، در درخت سورس اصلی کرنل ادغام می‌شود. سپس این اصلاحیه‌ها توسط توزیع‌کنندگان مختلف لینوکس (مانند Red Hat، اوبونتو، دبیان و غیره) در به‌روزرسانی‌های امنیتی کرنل برای کاربران نهایی منتشر می‌شوند. این اکوسیستم پیچیده از گزارش‌دهی، توسعه پچ، و توزیع، نشان‌دهنده تلاش مستمر برای حفظ امنیت کرنل است.

تنوع آسیب‌پذیری‌هایی که در زیرسیستم‌های مختلف کرنل – از جمله شبکه، فایل سیستم‌ها، درایورهای دستگاه، و مدیریت حافظه – کشف می‌شوند، گستردگی سطح حمله (attack surface) این نرم‌افزار پیچیده را به خوبی نشان می‌دهد. این امر چالش دائمی حفظ امنیت آن را برجسته‌تر می‌کند. علاوه بر این، انتشار اثبات مفهوم (Proof of Concept یا PoC) برای برخی از این آسیب‌پذیری‌ها، مانند مورد CVE-2024-53141، اهمیت و فوریت اعمال پچ‌های امنیتی را به شدت افزایش می‌دهد، زیرا یک آسیب‌پذیری نظری را به یک تهدید عملی و قابل بهره‌برداری تبدیل می‌کند. این پویایی، ماهیت بازی "موش و گربه" بین کشف آسیب‌پذیری، افشا، بهره‌برداری و اعمال پچ را در دنیای امنیت نرم‌افزار به تصویر می‌کشد.

۲.۲: تحلیل دقیق آسیب‌پذیری‌های منتخب اخیر (حداقل ۵ مورد)

در این بخش، به تحلیل دقیق پنج آسیب‌پذیری اخیراً کشف شده در کرنل لینوکس می‌پردازیم. برای هر آسیب‌پذیری، جزئیات فنی، تأثیر، تاریخ کشف و انتشار، و وضعیت رفع آن با استناد به منابع معتبر ارائه خواهد شد.

۱. CVE-2024-53141 : آسیب‌پذیری نوشتن خارج از محدوده (Out-of-Bounds Write) در

کامپوننت ipset کرنل

*شناسه و شرح CVE-2024-53141: یک آسیب‌پذیری جدی از نوع نوشتن خارج از محدوده در هیپ (kernel heap) است که در کامپوننت ipset مربوط به زیرسیستم netfilter کرنل لینوکس قرار دارد. این نقص امنیتی به مهاجمان اجازه می‌دهد تا با ارسال داده‌های دستکاری‌شده، کد دلخواه را با سطح دسترسی کرنل اجرا کنند.

*مولفه آسیب‌دیده و کد مربوطه: آسیب‌پذیری در تابع bitmap_ip_uadt واقع در فایل سورس net/netfilter/ipset/ip_set_bitmap_ip.c قرار دارد. مشکل زمانی بروز می‌کند که پارامتر IPSET_ATTR_CIDR (برای تعریف یک محدوده آدرس IP با استفاده از نماد CIDR) در درخواست ipset وجود داشته باشد، اما پارامتر IPSET_ATTR_IP_TO (که انتهای محدوده را مشخص می‌کند) ارائه نشده باشد. در این حالت، یک بررسی محدوده ضروری برای آدرس IP شروع، به درستی انجام نمی‌شود. قطعه کد آسیب‌پذیر را نشان داده شده است.

*جزئیات فنی: نقص اصلی در عدم وجود یک بررسی محدوده مناسب در تابع `bitmap_ip_uadt` است. هنگامی که شرایط خاصی در مورد صفات آدرس IP برآورده می‌شود (یعنی `tb` موجود نیست اما `tb` وجود دارد)، مقادیر `ip` (آدرس شروع) و `ip_to` (آدرس پایان) به درستی مدیریت نمی‌شوند یا محاسبات محدوده به گونه‌ای انجام می‌شود که منجر به نوشتن داده‌ها فراتر از مرزهای حافظه تخصیص داده شده برای `bitmap` می‌شود.

*تاثیر: این آسیب‌پذیری به مهاجم اجازه می‌دهد تا با دستکاری حافظه کرنل، کنترل اجرای برنامه را به دست گرفته و کد دلخواه را با بالاترین سطح دسترسی (سطح کرنل) اجرا کند، که به معنای افزایش سطح دسترسی (Privilege Escalation) کامل است. شدت این آسیب‌پذیری با امتیاز `CVSSv3.1` برابر با `۷.۸` (بالا) ارزیابی شده است.

*تاریخ کشف/انتشار: این آسیب‌پذیری در تاریخ ۶ دسامبر ۲۰۲۴ کشف شده است. ۱۷ یک اثبات مفهوم (PoC) برای بهره‌برداری از این آسیب‌پذیری نیز در حدود ۱۸ آوریل ۲۰۲۵ یا پس از آن به صورت عمومی منتشر شده است.

*تاریخ رفع و پیچ: این آسیب‌پذیری با کامیت (`commit`) به شماره هش `35f56c554eb1b56b77b3cf197a6b00922d49033d` در کد کرنل رفع شده است. این پیچ شامل افزودن بررسی‌های محدوده مناسب و حذف بررسی‌های غیرضروری در تابع `bitmap_ip_uadt` است تا از نوشتن خارج از محدوده جلوگیری شود. ۱۷ نسخه‌های اصلاح شده کرنل شامل نسخه‌های `۴.۱۹.۳۲۵`، `۶.۶.۶۴`، `۶.۱۱.۱۱`، و `۶.۱۲.۲` به بالا می‌باشند. ۱۷ (دسترسی مستقیم به جزئیات کامل این کامیت از طریق لینک‌های `git.kernel.org` در برخی موارد محدود بوده است ۱۷، اما شماره هش و هدف پیچ به طور گسترده گزارش شده است).

*منبع:

PoC Released for Linux Kernel Vulnerability Allowing Privilege ..., accessed June 1, 2025, <https://gbhackers.com/poc-released-for-linux-kernel-vulnerability/>

۲. CVE-2025-37980: آسیب‌پذیری نشت منابع (Resource Leak) در زیرسیستم بلاک کرنل

*شناسه و شرح: CVE-2025-37980 به یک آسیب‌پذیری نشت منابع در مسیر خطای تابع `blk_register_queue()` در زیرسیستم مدیریت دستگاه‌های بلاک (`block subsystem`) کرنل لینوکس اشاره دارد.

*مولفه آسیب‌دیده و کد مربوطه: تابع `blk_register_queue()` که برای ثبت یک صف جدید ورودی/خروجی برای یک دستگاه بلاک استفاده می‌شود. مشکل اصلی به دلیل عدم فراخوانی تابع `blk_mq_sysfs_unregister()` در مسیر مدیریت خطا، پس از موفقیت فراخوانی `blk_mq_sysfs_register()` و بروز خطای بعدی، رخ می‌دهد.

*جزئیات فنی: در صورتی که فرآیند ثبت یک صف جدید برای دستگاه بلاک، پس از اینکه منابع مربوط به `sysfs` برای صف چندگانه (multi-queue) با موفقیت توسط `blk_mq_sysfs_register()` تخصیص داده شد، با یک خطا در مراحل بعدی مواجه شود، کد مدیریت خطا به درستی منابع تخصیص داده شده توسط `sysfs` را آزاد نمی‌کند. این امر منجر به نشت حافظه (memory leak) می‌شود، زیرا این منابع تا زمان راه‌اندازی مجدد سیستم یا آزادسازی دستی (که معمولاً اتفاق نمی‌افتد) اشغال باقی می‌مانند.

*تأثیر: نشت حافظه در زیرسیستم بلاک کرنل. اگر این مسیر خطا به طور مکرر فعال شود، می‌تواند به تدریج منجر به اتمام منابع حافظه کرنل و در نهایت ناپایداری یا از کار افتادن سیستم شود.

*تاریخ کشف/انتشار: این آسیب‌پذیری در تاریخ ۲۰ می ۲۰۲۵ به طور عمومی افشا شده است.

*تاریخ رفع و پیچ: آسیب‌پذیری با افزودن فراخوانی تابع `blk_mq_sysfs_unregister()` به مسیر خطای مربوطه در `blk_register_queue()` رفع شده است. این تغییر تضمین می‌کند که در صورت بروز خطا، منابع `sysfs` به درستی پاک‌سازی می‌شوند. کامیت مربوط به این اصلاح با شماره هش `40f2eb9b531475dd01b683fdaf61ca3cfd03a51e` توسط Zheng Qixing در تاریخ شنبه، ۱۲ آوریل ۲۰۲۵، ساعت ۱۷:۲۵:۵۴ (به وقت چین، +۰۸۰۰) ایجاد شده است. ۲۳ (دسترسی مستقیم به جزئیات این کامیت از طریق لینک‌های `git.kernel.org` ممکن است محدود بوده باشد ۲۰، اما اطلاعات کامیت از طریق NVD و لاگ تغییرات کرنل تأیید شده است).

*منبع :

CVE-2025-37980 Impact, Exploitability, and Mitigation Steps | Wiz, accessed June 1, 2025, <https://www.wiz.io/vulnerability-database/cve/cve-2025-37980>

۳. CVE-2025-37896: آسیب پذیری خطای تقسیم (Divide Error) در زیرسیستم SPI کرنل

*شناسه و شرح CVE-2025-37896: یک آسیب پذیری از نوع خطای تقسیم (divide error) در زیرسیستم spi-mem (رابط حافظه SPI) کرنل لینوکس است. این خطا هنگام محاسبه مدت زمان عملیات حافظه SPI، در شرایطی که تعداد بایت های ساختگی (dummy bytes) صفر باشد، رخ می دهد.

*مولفه آسیب دیده و کد مربوطه: تابع spi_mem_calc_op_duration() که برای محاسبه زمان تخمینی یک عملیات بر روی حافظه SPI استفاده می شود. این خطا زمانی اتفاق می افتد که متغیر ncycles (تعداد سیکل های کلاک برای بایت های ساختگی) با مقدار صفر برای بایت های ساختگی محاسبه شود. برخی از عملیات حافظه فلش SPI، مانند دستورات write_cache و update_cache در دستگاه های حافظه فلش Winbond، SPINAND، نیازی به بایت ساختگی ندارند (یعنی تعداد بایت ساختگی آن ها صفر است).

*جزئیات فنی: تلاش برای محاسبه ncycles در تابع spi_mem_calc_op_duration() زمانی که تعداد بایت های ساختگی صفر است، منجر به یک عملیات تقسیم بر صفر می شود که یک خطای بحرانی در سطح کرنل ایجاد می کند. پشته خطای نمونه ارائه شده در گزارش ها: (Oops: divide error: 0000...? spi_mem_calc_op_duration+0x56/0xb0...) این موضوع را تأیید می کند.

*تأثیر: بروز این خطا منجر به توقف ناگهانی و از کار افتادن کرنل (Kernel panic) و در نتیجه از کار افتادن کل سیستم می شود.

*تاریخ کشف/انتشار: این آسیب پذیری در تاریخ ۲۰ می ۲۰۲۵ به طور عمومی منتشر شده است.

*تاریخ رفع و پیچ: این مشکل با اصلاح تابع spi_mem_calc_op_duration() به گونه ای که محاسبه ncycles برای مواردی که تعداد بایت های ساختگی صفر است، نادیده گرفته شود، رفع گردیده است. دو کامیت با شماره های هش 8e4d3d8a5e51e07bd0d6cdd81b5e4af79f796927 و 1915dbd67dad0bb35670c8e28229baa29368d17 به عنوان پیچ های مرتبط با این آسیب پذیری ذکر شده اند. ۲۷ تاریخ دقیق اعمال این کامیت ها در اسنپت ها به وضوح مشخص نیست، اما گزارش آسیب پذیری در تاریخ ۲۱ می ۲۰۲۵ به عنوان رفع شده به روزرسانی شده است. (دسترسی به جزئیات کامل این کامیت ها از طریق لینک های git.kernel.org در برخی موارد محدود بوده است).

*منبع:

CVE-2025-37896 : In the Linux kernel, the following vulnerability ..., accessed June 1, 2025, <https://www.cvedetails.com/cve/CVE-2025-37896/>

۴. CVE-2024-35965 : آسیب پذیری سرریز بافر (Buffer Overflow) در پیاده سازی L2CAP بلوتوث

کرنل

*شناسه و شرح CVE-2024-35965 : یک آسیب پذیری در پیاده سازی پروتکل L2CAP (Logical Link Control and Adaptation Protocol) در پشته بلوتوث کرنل لینوکس است. این آسیب پذیری ناشی از اعتبارسنجی ناکافی طول داده های ورودی کاربر در تابع `setsockopt` برای سوکت های L2CAP می باشد.

*مولفه آسیب دیده و کد مربوطه: تابع `setsockopt` در بخش مربوط به L2CAP در زیرسیستم بلوتوث کرنل (`net/bluetooth/l2cap_core.c` یا فایل های مشابه). عنوان کامیت مرتبط `"Bluetooth: L2CAP: Fix not validating setsockopt user input"` می باشد.

*جزئیات فنی: این نقص امنیتی به یک کاربر (یا برنامه در فضای کاربر) اجازه می دهد تا داده هایی با طول بسیار زیاد را از طریق فراخوانی `setsockopt` به کرنل ارسال کند. به دلیل عدم اعتبارسنجی کافی طول این داده ها، کرنل تلاش می کند این داده های حجیم را در یک بافر با اندازه محدود در فضای حافظه خود کپی کند. این عمل می تواند منجر به سرریز بافر (buffer overflow)، بازنویسی داده های مجاور در حافظه کرنل، و در نتیجه ناپایداری سیستم یا حتی اجرای کد مخرب شود.

*تاثیر: پیامدهای این آسیب پذیری می تواند شامل سرریز بافر، ناپایداری و از کار افتادن سیستم، و به طور بالقوه، افزایش سطح دسترسی مهاجم به سطح کرنل باشد. شرکت Red Hat شدت این آسیب پذیری را با امتیاز CVSSv3 پایه ۵.۵ (متوسط) ارزیابی کرده است.

*تاریخ کشف/انتشار: تاریخ انتشار اولیه این CVE در پایگاه داده NVD، ۲۰۲۴ می ۲۰ ذکر شده است ۳۳، اما با توجه به اینکه گزارش به آسیب پذیری های "اخیر" می پردازد و Red Hat اصلاحیه آن را (RHSA-2025:6966) برای RHEL 9 در تاریخ ۱۳ می ۲۰۲۵ منتشر کرده است ۱۵، و همچنین تاریخ به روزرسانی در برنامه CVE برابر با ۴ می ۲۰۲۵ است ۳۳، به نظر می رسد تاریخ های مرتبط با سال ۲۰۲۵ برای این گزارش مناسب تر باشند.

*تاریخ رفع و پیچ: این مشکل با افزودن بررسی های مناسب در تابع `setsockopt` برای اطمینان از اینکه اندازه داده های ورودی کاربر در محدوده های قابل قبول و امن قرار دارد، رفع شده است. ۱۵ باگزیلای مرتبط با این

آسیب‌پذیری در Red Hat شماره ۲۲۸۱۹۰۹ است. ۱۵ شماره هش کامیت دقیق پچ در اسنپت‌های موجود به طور صریح مشخص نشده است.

*منبع :

CVE-2024-35965 - Red Hat Customer Portal, accessed June 1, 2025,
<https://access.redhat.com/security/cve/cve-2024-35965>

۵. CVE-2024-23848 : آسیب‌پذیری استفاده پس از آزادسازی (Use-After-Free) در درایور CEC

کرنل

*شناسه و شرح CVE-2024-23848: یک آسیب‌پذیری از نوع استفاده پس از آزادسازی (Use-After-Free) در درایور CEC (Consumer Electronics Control) کرنل لینوکس است که نسخه‌های کرنل تا ۶.۷.۱ را تحت تأثیر قرار می‌دهد.

*مولفه آسیب‌دیده و کد مربوطه: این آسیب‌پذیری در تابع `cec_queue_msg_fh` که در فایل‌های `drivers/media/cec/core/cec-api.c` و `drivers/media/cec/core/cec-adap.c` قرار دارد، رخ می‌دهد.

*جزئیات فنی: مشکل اصلی یک وضعیت رقابتی (race condition) است. در این وضعیت، یک نخ (thread) در حال اجرای کرنل، قفل (lock) مربوط به یک ساختار داده `struct cec_fh` را به دست می‌آورد. همزمان، نخ دیگری ممکن است همان ساختار داده را آزاد (free) کند، بدون اینکه بررسی کند آیا نخ دیگری هنوز قفل آن را در اختیار دارد یا از آن استفاده می‌کند. این امر منجر به این می‌شود که نخ اول پس از آزادسازی حافظه، همچنان به آن (مثلاً به `fh->msgs`) دسترسی پیدا کند که یک خطای استفاده پس از آزادسازی کلاسیک است و می‌تواند منجر به خرابی حافظه شود.

*تأثیر: یک مهاجم محلی (local attacker) که قادر به تعامل با درایور CEC باشد، می‌تواند از این آسیب‌پذیری برای ایجاد شرایط انکار سرویس (Denial of Service) از طریق از کار انداختن سیستم (system crash) استفاده کند. همچنین، در برخی شرایط، این نوع آسیب‌پذیری‌ها می‌توانند منجر به اجرای کد دلخواه با سطح دسترسی کرنل شوند. ۱۳ شدت این آسیب‌پذیری با امتیاز CVSSv3.1 پایه ۵.۵ (متوسط) ارزیابی شده است.

*تاریخ کشف/انتشار: این آسیب‌پذیری در تاریخ ۲۳ ژانویه ۲۰۲۴ به طور عمومی منتشر شده است.

*تاریخ رفع و پچ: توزیع‌های مختلف لینوکس، از جمله اوبونتو (با انتشار نسخه‌های اصلاح شده مانند ۶.۸.۰-۴۴.۴۴ برای 24.04 LTS و ۵.۱۵.۰-۱۲۱.۱۳۱ برای 22.04 LTS و دیبیا (با انتشار نسخه‌های اصلاح شده مانند ۵.۱۰.۲۳۴-۱ برای bullseye و ۶.۱.۱۲۸-۱ برای bookworm) پچ‌های مربوط به این آسیب‌پذیری را ارائه کرده‌اند. شماره هش کامیت دقیق پچ در اسنپت‌های موجود به وضوح مشخص نشده است، اما به یک لینک از لیست پستی کرنل (LKML) اشاره شده است که احتمالاً حاوی اطلاعات پچ می‌باشد (هرچند دسترسی مستقیم به محتوای آن لینک در طول این تحقیق ممکن نبوده است).

*منبع:

Ubuntu alert USN-7539-1 (linux-raspi) - LWN.net, accessed June 1, 2025, <https://lwn.net/Articles/1022849/>

خلاصه‌ای از آسیب‌پذیری‌های اخیر منتخب در کرنل لینوکس

شناسه CVE	شرح مختصر	تاثیر اصلی	مولفه(های) آسیب‌دیده	کامیت رفع اشکال / نسخه(های) اصلاح شده	تاریخ کشف/انتشار
CVE-2024-53141	نوشتن خارج از محدوده در ipset (زیرسیستم netfilter)	اجرای کد دلخواه، افزایش سطح دسترسی	net/netfilter/ipset/ip_set_bitmap_ip.c (تابع bitmap_ip_uadt)	کامیت f56c554eb۳۵ / ...۱ نسخه‌های کرنل ۴.۱۹.۳۲۵، ۶.۶.۶۴، ۶.۱۱.۱۱، ۶.۱۲.۲ ¹⁷	۶ دسامبر ۲۰۲۴ (کشف) / ۱۸ آوریل ۲۰۲۵ (انتشار PoC)
CVE-2025-37980	نشت منابع در مسیر خطای blk_register_queue	اتمام منابع حافظه کرنل، ناپایداری سیستم	زیرسیستم بلاک (تابع blk_register_queue())	کامیت ۵۳۱f2eb9b۴۰ / ...۴ (توسط Zheng Qixing)	۲۰ می ۲۰۲۵ (افشا) / ۱۲ آوریل ۲۰۲۵ (تاریخ کامیت)

۲۰ می ۲۰۲۵ (انتشار) / ۲۱ می ۲۰۲۵ (به روزرسانی به عنوان رفع شده)	کامیت‌های ۵e4d3d8a5e۸ ...۱ و dbd67da۱۹۱۵ ...	drivers/spi/s pi-mem.c (تابع spi_mem_cal c_op_duratio n())	از کار افتادن کرنل (Kernel panic)	خطای تقسیم در زیرسیستم spi- mem	CVE-2025- 37896
۲۰ می ۲۰۲۴ (NVD) / ۱۳ می ۲۰۲۵ (رفع در Red Hat)	پچ شده در RHEL (RHSA- ۶۹۶۶:۲۰۲۵) و سایر توزیع‌ها	net/bluetoot h/l2cap_core .c (تابع setsockopt) / Bugzilla 2281909	سرریز بافر، ناپایداری، افزایش سطح دسترسی بالقوه	سرریز بافر در پایه‌سازی L2CAP بلوتوث	CVE-2024- 35965
۲۳ ژانویه ۲۰۲۴ (انتشار)	پچ شده در توزیع‌های مختلف لینوکس (مانند اوبونتو و دبیان)	drivers/medi a/cec/core/c ec-adap.c و cec-api.c (تابع cec_queue_ msg_fh)	انکار سرویس (DoS)، اجرای کد دلخواه بالقوه	استفاده پس از آزادسازی در درایور CEC کرنل	CVE-2024- 23848

این جدول، نمایی کلی از آسیب‌پذیری‌های مورد بررسی ارائه می‌دهد و به درک سریع‌تر گستردگی و تنوع مشکلات امنیتی که می‌توانند در کرنل لینوکس بروز کنند، کمک می‌کند. مشاهده می‌شود که این آسیب‌پذیری‌ها بخش‌های مختلفی از کرنل، از جمله شبکه، مدیریت دستگاه‌ها و پروتکل‌های ارتباطی را تحت تأثیر قرار می‌دهند.

ج) در این بخش می خواهیم با کاربرد ساده ولی بسیار مهمی آشنا شویم. با استفاده از دستورات لینوکس، باید این کار را انجام بدهید. ابتدا یک فایل **text** خالی را با نام **EmbeddedGoogle** در گوگل درایو خود قرار بدهید. سپس آن را دانلود کنید، سپس زمان دقیق آن لحظه را با دستورات مربوطه به فرمت مناسب (تاریخ شامل روز، ماه، سال و همچنین زمان شامل ساعت، دقیقه و ثانیه) را وارد کنید به همراه نام و نام خانوادگی و شماره دانشجویی تان و شماره ای که با آن عضو کانال رسمی درس هستید به همراه آدرس جیمیل تان که مرتبط با گوگل درایو تان است (در خطوط جداگانه) سپس آن را مجدد در گوگل درایو خود آپلود نمائید. آن را **Access Open** کنید و لینک آن را در گزارش بگذارید. (در گزارش تان حتما تصاویر کافی به همراه توضیحات از تمام مراحل کار باشد).

ابتدا برای ارتباط مستقیم با گوگل درایو از طریق خط فرمان، ابزار **rclone** را نصب می کنیم:

`sudo apt update`

`sudo apt install rclone`

```
meysam@ubuntu:~/Desktop$ sudo apt install rclone
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  rclone
0 upgraded, 1 newly installed, 0 to remove and 501 not upgraded.
Need to get 10.6 MB of archives.
After this operation, 42.2 MB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu focal-updates/universe amd64 rclone amd64 1.50.2-2ubuntu0.2 [10.6 MB]
Fetched 10.6 MB in 8s (1,301 kB/s)
Selecting previously unselected package rclone.
(Reading database ... 123628 files and directories currently installed.)
Preparing to unpack .../rclone_1.50.2-2ubuntu0.2_amd64.deb ...
Unpacking rclone (1.50.2-2ubuntu0.2) ...
Setting up rclone (1.50.2-2ubuntu0.2) ...
Processing triggers for man-db (2.9.1-1) ...
```

برای اینکه بتوانیم از طریق خط فرمان به فضای گوگل درایو دسترسی داشته باشیم، باید یک **remote** جدید در **rclone** بسازیم:

`rclone config`

پس از اجرای دستور، منویی مشابه زیر نمایش داده می شود:

```
meysam@ubuntu:~/Desktop$ rclone config
2025/05/29 12:17:48 NOTICE: Config file "/home/meysam/.config/rclone/rclone.conf" not found - using defaults
No remotes found - make a new one
n) New remote
s) Set configuration password
q) Quit config
n/s/q> n
```

با زدن گزینه ی **n**، یک **remote** جدید ایجاد می کنیم و نام دلخواه مثلاً **meysam** را وارد می کنیم.

در مرحله‌ی بعد از لیست سرویس‌ها، گزینه‌ی مربوط به Google Drive (معمولاً عدد ۱۳ بسته به نسخه‌ی rclone) انتخاب می‌شود.

```
13 / Google Drive
   \ "drive"
14 / Google Photos
   \ "google photos"
15 / Hubic
   \ "hubic"
16 / JottaCloud
   \ "jottacloud"
17 / Koofr
   \ "koofr"
18 / Local Disk
   \ "local"
19 / Mail.ru Cloud
   \ "mailru"
20 / Microsoft Azure Blob Storage
   \ "azureblob"
21 / Microsoft OneDrive
   \ "onedrive"
22 / OpenDrive
   \ "opendrive"
23 / Openstack Swift (Rackspace Cloud Files, Memset Memstore, OVH)
   \ "swift"
24 / Pcloud
   \ "pcloud"
25 / Put.io
   \ "putio"
26 / SSH/SFTP Connection
   \ "sftp"
27 / Transparently chunk/split large files
   \ "chunker"
28 / Union merges the contents of several remotes
   \ "union"
29 / Webdav
   \ "webdav"
30 / Yandex Disk
   \ "yandex"
31 / http Connection
   \ "http"
32 / premiumize.me
   \ "premiumizeme"
Storage> 13
```

باقی مراحل (OAuth، انتخاب حساب گوگل، اعطای دسترسی و...) با دنبال کردن دستورالعمل‌های نمایش داده شده انجام می‌شود تا اینکه در نهایت پیغام زیر نمایش داده شود:

```
* Say N if you are working on a remote or headless machine
y) Yes
n) No
y/n>
y/n> y
If your browser doesn't open automatically go to the following link: http://127.0.0.1:53682/auth?state=dOztXtDz_RVLQIJPSMJW7w
Log in and authorize rclone for access
Waiting for code...
Got code
Configure this as a team drive?
y) Yes
n) No
y/n> y
Fetching team drive list...
No team drives found in your account-----
[meysam]
scope = drive
token = {"access_token":"ya29.a0AW4XtXifgF341_SHLJnb9-tBfbovEMisTd45hUnZH-413uS2Ly9MZvUUD2761A5ro77NGzEbxX_wRnuUXK7EiF0Jp2gBM
YKAcwSARASFQHGx2Mi7tGGFzm5MyPn8GJvxwgLKg0175","token_type":"Bearer","refresh_token":"1//09XiARMFJ6SN0CgYIARAAGAkSNwF-L9IrPe_Ir
5-05-29T13:20:51.42998305-07:00"}
team_drive =
-----
y) Yes this is OK
e) Edit this remote
d) Delete this remote
y/e/d> y
Current remotes:

Name                Type
====
meysam              drive

e) Edit existing remote
n) New remote
d) Delete remote
r) Rename remote
c) Copy remote
s) Set configuration password
q) Quit config
e/n/d/r/c/s/q> q
```

سپس یک فایل متنی خالی با نام EmbeddedGoogle.txt ایجاد می‌کنیم:

```
touch EmbeddedGoogle.txt
```

با استفاده از دستور `rclone copy` فایل خالی را به ریموت `meysam` در گوگل درایو منتقل می‌کنیم:

```
rclone copy EmbeddedGoogle.txt meysam:/EmbeddedGoogle.txt
```

دستور به‌صورت بدون خطا اجرا می‌شود و فایل در گوگل درایو بارگذاری می‌گردد. برای اطمینان از وجود فایل بارگذاری‌شده، لیست پوشه اصلی در گوگل درایو را مشاهده می‌کنیم:

```
rclone ls meysam:
```

خروجی‌ای مشابه زیر نمایش داده می‌شود:

```
0 EmbeddedGoogle.txt
```

برای ویرایش محتوا، ابتدا فایل را از گوگل درایو به سیستم محلی خود دانلود می‌کنیم:

```
rclose copy meysam:/EmbeddedGoogle.txt ./
```

برای ثبت زمان دقیق لحظه جاری (روز/ماه/سال ساعت:دقیقه:ثانیه)، از دستور **date** استفاده می‌کنیم:

```
date '+%Y-%m-%d %H:%M:%S'
```

طبق صورت تمرین، اطلاعات زیر باید در خطوط جداگانه به ابتدای فایل اضافه شوند:

۱. **Timestamp**: زمان دقیق (خروجی **date**)

۲. **First Name**: محمد میثم (یا به انگلیسی Mohammad Meysam)

۳. **Last Name**: الهام‌بخش (Elhambakhsh)

۴. **Student ID**: 403301526

۵. **Phone Number**: شماره تماسی که با آن در کلنال رسمی درس عضو شده‌ایم (اینجا:

+۹۸۹۱۰۲۳۰۶۰۹۳)

۶. **Email**: آدرس جیمیل مرتبط با گوگل درایو (meysam.inspiring@gmail.com)

برای اینکه اطلاعات جدید را به ابتدای فایل اصلی اضافه کنیم، از یک فایل موقت **tmp.txt** استفاده می‌کنیم. مراحل زیر را طی می‌کنیم:

۱. تعریف متغیر **Timestamp** و قرار دادن سایر اطلاعات در فایل موقت

```
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
```

```
cat <<EOF > tmp.txt
```

```
Timestamp: $TIMESTAMP
```

```
First Name: Mohammad Meysam
```

```
Last Name: Elhambakhsh
```

```
Student ID: 403301526
```

```
Phone Number: +989102306093
```

```
Email: meysam.inspiring@gmail.com
```

EOF

○ در اینجا، با استفاده از دستور (...) مقدار خروجی دستور date را در متغیر TIMESTAMP ذخیره می‌کنیم.

○ سپس با استفاده از EOF ... EOF <<cat محتوای موردنظر را به tmp.txt می‌نویسیم.

۲. ترکیب محتوای فایل موقت با فایل اصلی و ایجاد فایل جدید

```
cat tmp.txt EmbeddedGoogle.txt > EmbeddedGoogle_updated.txt
```

○ در این مرحله، ابتدا محتوای tmp.txt (اطلاعات شخصی) و سپس محتوای قبلی EmbeddedGoogle.txt (که در حال حاضر خالی است یا حاوی اطلاعات قبلی فایل) را درون EmbeddedGoogle_updated.txt می‌ریزیم.

۳. جایگزینی فایل اصلی با نسخه به‌روزشده

```
mv EmbeddedGoogle_updated.txt EmbeddedGoogle.txt
```

○ با دستور بالا، فایل جدید را جای فایل اصلی می‌گذاریم تا نام ثابت بماند (EmbeddedGoogle.txt).

○ سپس فایل موقت tmp.txt را حذف می‌کنیم تا محیط کاری تمیز شود:

```
rm tmp.txt
```

۴. بررسی نهایی محتویات فایل

```
cat EmbeddedGoogle.txt
```

○ با اجرای این دستور، باید خروجی شبیه به زیر را مشاهده کنیم:

Timestamp: 2025-05-29 12:27:48

First Name: Mohammad Meysam

Last Name: Elhambakhsh

Student ID: 403301526

Phone Number: +989102306093

Email: meysam.inspiring@gmail.com

برای اینکه فایل جدید را با اطلاعات اضافه‌شده، مجدداً در گوگل درایو بارگذاری کنیم، از گزینه `--update` استفاده می‌کنیم تا فقط در صورت تغییر فایل آپلود انجام شود:

```
rclone copy EmbeddedGoogle.txt meysam:/EmbeddedGoogle.txt --update
```

```
meysam@ubuntu:~/Desktop$ rclone copy EmbeddedGoogle.txt meysam:/EmbeddedGoogle.txt \
> --update
```

برای به‌دست آوردن لینک دسترسی عمومی (Access Open)، از دستور زیر استفاده می‌کنیم:

```
rclone link meysam:/EmbeddedGoogle.txt
```

به این ترتیب لینک زیر ایجاد شد که به وسیله آن می‌توانید خروجی را مشاهده فرمایید.

<https://drive.google.com/open?id=1DLJuPkYBWcKrzSE6jqQWrwL2CnK5SUx5>

```
meysam@ubuntu:~/Desktop$ rclone link meysam:/EmbeddedGoogle.txt
https://drive.google.com/open?id=1DLJuPkYBWcKrzSE6jqQWrwL2CnK5SUx5
```

تمرین remote monitor:

هدف این تمرین، طراحی و پیاده‌سازی یک برنامه مانیتورینگ از راه دور (Remote Monitor) با استفاده از زبان برنامه‌نویسی C++ است. این برنامه قادر است از طریق پروتکل SSH به یک سیستم لینوکسی دیگر متصل شده، اطلاعات سخت‌افزاری آن (پردازنده و حافظه) را نمایش دهد و به‌صورت دوره‌ای (هر ۳۰ ثانیه) میزان استفاده از پردازنده و حافظه سیستم مقصد را بررسی کند. در صورتی که این مقادیر از آستانه‌های تعریف‌شده توسط کاربر فراتر روند، برنامه ضمن نمایش هشدار، پراسسی که بیشترین سهم را در این افزایش داشته شناسایی کرده، ID آن را نمایش داده و سپس آن را متوقف می‌کند. برای ساخت و مدیریت پروژه از ابزارهای make و Cmake استفاده شده است.

ساختار پروژه

پروژه دارای ساختار دایرکتوری و فایل‌های زیر است که به سازماندهی بهتر کد و فرآیند ساخت کمک می‌کند:

remote_monitor/

| — src/

| | — main.cpp

| | — ssh_connection.cpp

| | — hardware_monitor.cpp

| — include/

| | — ssh_connection.h

| | — hardware_monitor.h

| — CMakeLists.txt

| — Makefile

- build/: این دایرکتوری پس از اجرای CMake و کامپایل ایجاد می‌شود
- src/: شامل فایل‌های سورس کد (.cpp) برنامه است.
- include/: شامل فایل‌های هدر (.h) برای تعاریف کلاس‌ها و توابع است.
- CMakeLists.txt: فایل پیکربندی برای Cmake جهت ساخت پروژه.

- **Makefile**: یک فایل Makefile سنتی برای کامپایل پروژه (می‌تواند به عنوان جایگزین یا مکمل CMake استفاده شود).
- **build/**: دایرکتوری که فایل‌های ساخته شده توسط Cmake و کامپایلر در آن قرار می‌گیرند (معمولاً به صورت out-of-source build).

اجزای اصلی برنامه

برنامه از سه جزء اصلی تشکیل شده است: مدیریت اتصال SSH، مانیتورینگ سخت‌افزار، و منطق اصلی برنامه.

۱. مدیریت اتصال (ssh_connection.h, ssh_connection.cpp)

این کلاس مسئول برقراری و مدیریت اتصال SSH به سیستم ریموت است.

- **SSHConnection::SSHConnection(host, user, password)**: سازنده کلاس که اطلاعات لازم برای اتصال (میزبان، نام کاربری، رمز عبور) را دریافت می‌کند.
- **SSHConnection::~~SSHConnection()**: مخرب کلاس که در صورت وجود اتصال، آن را قطع کرده و منابع تخصیص داده شده به libssh را آزاد می‌کند.
- **bool SSHConnection::connect()**: این متد تلاش می‌کند تا با استفاده از کتابخانه libssh یک جلسه SSH جدید ایجاد کند، گزینه‌های مربوط به میزبان و کاربر را تنظیم نماید، به هاست متصل شود و سپس با استفاده از رمز عبور، احراز هویت کاربر را انجام دهد. در صورت بروز هرگونه خطا، پیام مناسب چاپ شده و false برگردانده می‌شود.
- **std::string SSHConnection::executeCommand(command)**: این متد یک دستور مشخص را از طریق کانال SSH بر روی سیستم ریموت اجرا کرده و خروجی استاندارد آن دستور را به صورت یک رشته متنی برمی‌گرداند. این تابع برای اجرای دستوراتی مانند `ls`, `cpu`, `free`, `ip`, `addr`, `kill` و `top`, `ps` استفاده می‌شود.

۲. مانیتورینگ سخت‌افزار (hardware_monitor.h, hardware_monitor.cpp)

این کلاس از نمونه‌ای از SSHConnection برای اجرای دستورات روی سیستم ریموت و جمع‌آوری اطلاعات سخت‌افزاری استفاده می‌کند.

- **HardwareMonitor::HardwareMonitor(connection)**: سازنده کلاس که یک ارجاع به شیء SSHConnection فعال را دریافت می‌کند.
- **std::string HardwareMonitor::executeAndGetOutput(command)**: یک متد کمکی خصوصی که صرفاً `connection.executeCommand(command)` را فراخوانی می‌کند.
- **void HardwareMonitor::displayHardwareInfo()**: این متد اطلاعات اولیه سخت‌افزاری سیستم ریموت را نمایش می‌دهد:
 - **اطلاعات پردازنده**: با اجرای دستور `lscpu` و تجزیه خروجی آن، معماری، تعداد پردازنده‌ها و نام مدل پردازنده نمایش داده می‌شود.
 - **اطلاعات حافظه**: با اجرای `free -h | grep Mem`، اطلاعات کلی حافظه (کل، استفاده‌شده، آزاد و ...) نمایش داده می‌شود.
 - **اطلاعات اتصال SSH**: با اجرای `ip addr show wlan0 | grep inet`، آدرس IP سیستم ریموت روی اینترفیس `wlan0` نمایش داده می‌شود.
- **double HardwareMonitor::getCPUUsage()**: این متد با اجرای دستور `top -bn1 | grep 'Cpu(s)'` و تجزیه خروجی آن، درصد استفاده لحظه‌ای کاربر از CPU (us) را استخراج و برمی‌گرداند.
- **double HardwareMonitor::getMemoryUsage()**: با اجرای دستور `free -m | grep Mem` و تجزیه خروجی، درصد کل حافظه استفاده‌شده را محاسبه و برمی‌گرداند.
- **std::pair<int, double> HardwareMonitor::getTopCPUProcess()**: با اجرای دستور `ps -eo pid,pcpu --sort=-pcpu | head -n 2` و درصد استفاده از CPU پراسسی که بیشترین بار را روی CPU دارد، استخراج می‌کند.
- **std::pair<int, double> HardwareMonitor::getTopMemoryProcess()**: با اجرای دستور `ps -eo pid,pmem --sort=-pmem | head -n 2` و درصد استفاده از حافظه پراسسی که بیشترین حافظه را مصرف می‌کند، استخراج می‌کند.

۳. برنامه اصلی (main.cpp)

این فایل نقطه ورود برنامه است و وظایف زیر را انجام می‌دهد:

- **دریافت آرگومان‌های ورودی:** برنامه هنگام اجرا، آدرس میزبان (به فرمت user@host)، رمز عبور، آستانه استفاده از CPU (درصد) و آستانه استفاده از حافظه (درصد) را به عنوان آرگومان‌های خط فرمان دریافت می‌کند.

`./remote_monitor <user@host> <password> <cpu_threshold> <mem_threshold>`

- **برقراری اتصال:** یک شیء SSHConnection ایجاد کرده و تلاش می‌کند به سیستم ریموت متصل شود. در صورت عدم موفقیت، برنامه با خطا خاتمه می‌یابد.
- **نمایش اطلاعات اولیه:** پس از اتصال موفق، اطلاعات سخت‌افزاری اولیه سیستم ریموت توسط `HardwareMonitor::displayHardwareInfo()` نمایش داده می‌شود.
- **حلقه مانیتورینگ:** برنامه وارد یک حلقه بی‌نهایت می‌شود که در هر تکرار (هر ۳۰ ثانیه):

۱. زمان و تاریخ فعلی را چاپ می‌کند.

۲. درصد استفاده از CPU و حافظه را با استفاده از متدهای `HardwareMonitor` دریافت و نمایش می‌دهد.

۳. بررسی آستانه CPU: اگر درصد استفاده از CPU از آستانه تعریف‌شده بیشتر باشد:

- پیام هشدار نمایش داده می‌شود.
- پراسسی که بیشترین CPU را مصرف می‌کند، شناسایی شده و PID آن نمایش داده می‌شود.

▪ دستور `kill` برای متوقف کردن آن پراسس به سیستم ریموت ارسال می‌شود.

۴. بررسی آستانه حافظه: اگر درصد استفاده از حافظه از آستانه تعریف‌شده بیشتر باشد:

- پیام هشدار نمایش داده می‌شود.
- پراسسی که بیشترین حافظه را مصرف می‌کند، شناسایی شده و PID آن نمایش داده می‌شود.

▪ دستور `kill` برای متوقف کردن آن پراسس به سیستم ریموت ارسال می‌شود.

۵. برنامه به مدت ۳۰ ثانیه متوقف می‌شود. (std::this_thread::sleep_for)

سیستم ساخت (Build System)

برای کامپایل و ساخت پروژه از Cmake و یک Makefile دستی استفاده شده است.

۱. CMakeLists.txt

این فایل وظیفه پیکربندی فرآیند ساخت پروژه را با استفاده از CMake بر عهده دارد.

```
cmake_minimum_required(VERSION 3.10)
```

#تعیین حداقل نسخه CMake مورد نیاز

```
project(RemoteMonitor)
```

#نام پروژه

```
set(CMAKE_CXX_STANDARD 11)
```

#تنظیم استاندارد C++ به C++11

```
include_directories(include)
```

#افزودن دایرکتوری include به مسیرهای جستجوی هدر فایل‌ها

```
find_package(libssh REQUIRED)
```

#جستجو و یافتن کتابخانه libssh که برای عملکرد برنامه ضروری است

```
add_executable(remote_monitor
```

#تعریف فایل اجرایی نهایی با نام remote_monitor

```
src/main.cpp
```

```
src/ssh_connection.cpp
```

```
src/hardware_monitor.cpp
```

```
)
```

```
target_link_libraries(remote_monitor ssh)
```

#لینک کردن فایل اجرایی با کتابخانه libssh (ssh)

نحوه پاک‌سازی و ساخت مجدد با **Cmake**: معمولاً با Cmake به روش "out-of-source build" کار می‌کنند. یعنی یک دایرکتوری جداگانه (مثلاً build) ایجاد کرده، وارد آن شده و دستورات cmake و make را در آنجا اجرا می‌کنند:

```
mkdir build
```

```
cd build
```

```
cmake ..
```

#به دایرکتوری اصلی پروژه که CMakeLists.txt در آن قرار دارد، اشاره می‌کند.

```
make
```

برای پاک‌سازی، کافی است محتویات دایرکتوری build حذف شود یا دستور make clean (اگر توسط cmake ایجاد شده باشد) در داخل دایرکتوری build اجرا شود. اگر می‌خواهید اطمینان حاصل کنید که قبل از هر ساخت جدید، فایل‌های قبلی پاک می‌شوند، می‌توانید قبل از اجرای .. cmake دایرکتوری build را حذف و دوباره ایجاد کنید.

۲. Makefile

فایل Makefile ارائه شده یک روش سنتی برای ساخت پروژه با استفاده از ابزار make است.

```
CC = g++
```

#تعریف کامپایلر C++

```
CFLAGS = -Wall -Wextra -linclude
```

#فلگ‌های کامپایلر: نمایش همه هشدارها، هشدارهای اضافی، مسیر دایرکتوری include

```
LDFLAGS = -lssh
```

#فلگ‌های لینکر: لینک با کتابخانه libssh

```
SRCS = src/main.cpp src/ssh_connection.cpp src/hardware_monitor.cpp
```

#لیست فایل‌های سورس

OBJS = \$(SRCS:.cpp=.o)

#تبدیل لیست فایل‌های سورس به لیست فایل‌های آبجکت (.o)

TARGET = remote_monitor

#نام فایل اجرایی نهایی

all: \$(TARGET)

#هدف پیش‌فرض: ساخت فایل اجرایی

\$(TARGET): \$(OBJS)

#قانون ساخت فایل اجرایی: وابسته به فایل‌های آبجکت

\$(CC) \$(OBJS) -o \$(TARGET) \$(LDFLAGS)

#دستور لینک کردن فایل‌های آبجکت

%.o: %.cpp

#قانون ساخت فایل‌های آبجکت از فایل‌های سورس متناظر

\$(CC) \$(CFLAGS) -c \$< -o \$@

#دستور کامپایل کردن یک فایل سورس به آبجکت

clean:

#هدف برای پاک‌سازی

rm -f \$(OBJS) \$(TARGET)

#حذف فایل‌های آبجکت و فایل اجرایی

.PHONY: all clean

#تعریف اهداف مجازی (Phony targets)

- برای کامپایل پروژه: make یا make all
- برای پاک‌سازی فایل‌های ساخته‌شده: make clean

نحوه اجرا و استفاده

۱. پیش‌نیازها:

کامپایلر C++ (مانند g++)، کتابخانه libssh و هدرهای توسعه آن (libssh-dev) یا معادل آن در توزیع لینوکس، ابزار Cmake (اگر از CMakeLists.txt استفاده می‌کنید)، ابزار make

۲. نصب libssh (در صورت نیاز):

```
sudo apt-get update sudo apt-get install libssh-dev
```

۳. کامپایل پروژه:

- با استفاده از **Cmake** : `bash #` در دایرکتوری ریشه پروژه `mkdir -p (remote_monitor)`
`make .. build` `cd build` `cmake` `build` در دایرکتوری `remote_monitor` فایل اجرایی ایجاد می‌شود.

- با استفاده از **Makefile** دستی: در دایرکتوری ریشه پروژه `(remote_monitor)` با اجرای فرمان `make` فایل اجرایی `remote_monitor` در دایرکتوری ریشه پروژه ایجاد می‌شود.

۴. اجرای برنامه :برنامه را با آرگومان‌های مورد نیاز اجرا کنید:

اگر از CMake استفاده کرده‌اید:

```
./build/remote_monitor user@remote_host_ip "your_password"  
cpu_threshold_percent memory_threshold_percent
```

مثال:

```
./build/remote_monitor myuser@192.168.1.10 "p@$wOrd" 70 60
```

اگر از Makefile دستی استفاده کرده‌اید:

```
./remote_monitor user@remote_host_ip "your_password"  
cpu_threshold_percent memory_threshold_percent
```

مثال:

```
./remote_monitor myuser@192.168.1.10 "p@$wOrd" 70 60
```

- ``user@remote_host_ip``: نام کاربری و آدرس IP یا نام هاست سیستم ریموت.
- ``"your_password"``: رمز عبور کاربر در سیستم ریموت (در صورت داشتن کاراکترهای خاص، آن را داخل دابل کوتیشن قرار دهید).
- ``cpu_threshold_percent``: آستانه استفاده از CPU (مثلاً ``۷۰`` برای `۷۰٪`).
- ``memory_threshold_percent``: آستانه استفاده از حافظه (مثلاً ``۶۰`` برای `۶۰٪`).

خروجی و نمایش عملکرد

خروجی برنامه مشابه نمونه ارائه شده در صورت سوال خواهد بود. ابتدا اطلاعات سیستم ریموت و سپس در هر ۳۰ ثانیه، میزان استفاده از CPU و حافظه نمایش داده می‌شود. در صورت عبور از آستانه‌ها، هشدارهای مربوطه به همراه PID پراسس خاطی و عملیات kill نمایش داده خواهد شد.

خروجی برنامه

```
meysam@ubuntu:~/Desktop/remote_monitor/build$ ./remote_monitor root@192.168.222.100 orangepi 100 23
Connecting to remote host...
SSH connection established and authenticated successfully!
Connection established.
System CPU Information:
Architecture:          aarch64
CPU(s):                4
Model name:             Cortex-A53
NUMA node0 CPU(s):     0-3

System Memory Information:
Mem:          477Mi      331Mi      85Mi      1.0Mi      60Mi      134Mi

SSH Connection Information (eth0):

[09:39:13 30-05-2025]
CPU Usage: 47.1%
Memory Usage: 51.2%
Warning: Memory usage exceeds threshold!
Killing process with PID: 15878 due to high memory usage (22.1%).

[09:39:44 30-05-2025]
CPU Usage: 1.2%
Memory Usage: 22.0%
```

این پروژه با موفقیت یک ابزار مانیتورینگ از راه دور را با استفاده از C++ و libssh پیاده‌سازی می‌کند. برنامه قادر است به سیستم‌های لینوکسی دیگر متصل شده، منابع سخت‌افزاری آن‌ها را پایش کرده و در صورت استفاده بیش از حد از منابع، اقدامات لازم را انجام دهد. استفاده از Cmake و Makefile نیز فرآیند ساخت و مدیریت پروژه را تسهیل می‌بخشد. نمایش خوانا و کاربرپسند اطلاعات و همچنین عملکرد صحیح در شرایط مختلف، از نقاط قوت این پروژه است.

تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

تمرین دوم : راه اندازی برد OrangePi Zero 2 Plus

اتصال به Wi-Fi و SSH

ایجاد Hotspot وای فای

ابتدا، یک شبکه Wi-Fi با استفاده از تلفن همراه یا لپ تاپ ایجاد می کنیم. مشخصات این شبکه برای استفاده در اسکریپت ضروری است.

```
nmcli device wifi connect "$SSID" password "$PASSWORD"
```

- **SSID** (نام شبکه): Meysam1 (مطابق با اسکریپت)
- **Password** (رمز عبور): HoE@1234 (مطابق با اسکریپت)
- **Frequency** (فرکانس): 2.4 GHz

نکته: دسترسی به اینترنت در این شبکه باید فعال باشد تا در صورت نیاز به دانلود بسته ها یا سایر ارتباطات، مشکلی وجود نداشته باشد.

اتصال به برد از طریق SSH

بعد از اتصال به وای فای ابتدا با کامند زیر اتصال را چک می کنیم.

```
nmcli device wifi list
```

```
orangepi@orangepizero2h5:~$ nmcli device wifi list
```

IN-USE	BSSID	SSID	MODE	CHAN	RATE	SIGNAL	BARS	SECURITY
*	52:4E:5C:86:47:1F	Meysam1	Infra	6	130 Mbit/s	57		WPA2
	A8:E5:44:A4:4B:79	IranCell-TF-i60-4B79_1	Infra	10	270 Mbit/s	40		WPA2
	98:9C:57:C8:7F:43	--	Infra	9	130 Mbit/s	37		WPA1 WPA2
	98:9C:57:C8:7F:42	--	Infra	9	130 Mbit/s	34		WPA1 WPA2

اگر اتصال به درستی انجام شده بود، به کمک دستور `ifconfig` آیپی دستگاه را یادداشت کرده سپس سیستم ثانویه که می خواهیم از آن به `orangepi` ارتباط `ssh` ایجاد کنیم هم به همین شبکه وصل کرده دستور زیر را اجرا می کنیم.

```
ssh orangepi@ip
```


برای راحتی کار و مدیریت برد، از یک کلاینت SSH مانند PuTTY یا افزونه Remote – SSH در ویرایشگر VS Code استفاده می‌کنیم. این کار به ما اجازه می‌دهد تا دستورات را به راحتی در ترمینال برد اجرا کرده، فایل‌ها را ویرایش و اسکریپت‌ها را مدیریت کنیم.

تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

اتصال خودکار به Wi-Fi

برای مدیریت اتصال خودکار، اسکریپت auto-connect.sh را ایجاد کرده‌ایم. این اسکریپت وظیفه بررسی و برقراری اتصال را بر عهده دارد.

محتوای اسکریپت

محتوای اسکریپت ارائه شده به شرح زیر است:

```
#!/bin/bash
# Wi-Fi credentials
SSID="Meysam1"
PASSWORD="HoE@1234"
LOG_FILE="/var/log/auto-connect.log"
# Ensure log file is writable
touch $LOG_FILE
chmod 666 $LOG_FILE
# Check if already connected
if nmcli -t -f NAME connection show --active | grep -q "$SSID"; then
    echo "$(date) - Already connected to $SSID" >> $LOG_FILE
else
    echo "$(date) - Not connected to $SSID. Attempting to connect..." >> $LOG_FILE
    nmcli device wifi connect "$SSID" password "$PASSWORD" >> $LOG_FILE 2>&1
```

```

if [ $? -eq 0 ]; then
    echo "$(date) - Successfully connected to $SSID" >> $LOG_FILE
else
    echo "$(date) - Failed to connect to $SSID" >> $LOG_FILE
fi
fi
# Log available networks for debugging
echo "$(date) - Available Wi-Fi networks:" >> $LOG_FILE
nmcli device wifi list >> $LOG_FILE 2>&1

```

توضیح عملکرد اسکریپت

- **SSID="Meysam1" و PASSWORD="HoE@1234"** : در این دو خط، نام و رمز عبور شبکه‌ی Wi-Fi مورد نظر تعریف شده است .
- **LOG_FILE="/var/log/auto-connect.log"** : مسیر فایل لاگ که تمام رویدادها در آن ثبت می‌شوند، مشخص شده است .
- **touch \$LOG_FILE و chmod 666 \$LOG_FILE** : این دو دستور اطمینان حاصل می‌کنند که فایل لاگ وجود دارد و قابل نوشتن است .
- **if nmcli -t -f NAME connection show --active | grep -q "\$SSID"** : این بخش هسته اصلی اسکریپت است. با استفاده از دستور nmcli، لیست اتصالات فعال بررسی می‌شود. اگر نام شبکه (SSID) در لیست اتصالات فعال وجود داشته باشد، به این معنی است که دستگاه متصل است. در این حالت، یک پیام "Already connected" در فایل لاگ ثبت می‌شود .
- **else ...** : اگر دستگاه متصل نباشد، اسکریپت وارد این بخش می‌شود. ابتدا یک پیام مبنی بر تلاش برای اتصال در فایل لاگ ثبت می‌کند .
- **nmcli device wifi connect "\$SSID" password "\$PASSWORD"** : این دستور اصلی‌ترین بخش عملیات است و تلاش می‌کند با استفاده از نام و رمز عبور مشخص‌شده، به شبکه

WiFi متصل شود. خروجی استاندارد و خطای این دستور به فایل لاگ هدایت می‌شود
(>>\$LOG_FILE 2>&1).

- **if [\$? -eq 0]**: پس از اجرای دستور اتصال، وضعیت خروجی آن (**\$?**) بررسی می‌شود. اگر مقدار آن 0 باشد، به معنی موفقیت‌آمیز بودن اتصال است و پیام "Successfully connected" در لاگ ثبت می‌شود. در غیر این صورت، پیام "Failed to connect" ثبت می‌گردد.

- **nmcli device wifi list**: در انتهای اسکریپت، لیست تمام شبکه‌های Wi-Fi موجود در اطراف اسکن شده و برای اهداف عیب‌یابی در فایل لاگ ثبت می‌شود.

آماده‌سازی اسکریپت

قبل از تنظیم cron job، باید اسکریپت را در یک مسیر مشخص

مثلا (/home/user/scripts/auto-connect.sh) قرار داده و آن را اجرایی کنیم:

```
chmod +x /home/user/scripts/auto-connect.sh
```

خودکارسازی اسکریپت با cron

Cron یک ابزار زمان‌بندی وظایف در سیستم‌عامل‌های لینوکسی است که به ما اجازه می‌دهد اسکریپت auto-connect.sh را به صورت خودکار اجرا کنیم.

ویرایش Crontab

برای افزودن وظایف زمان‌بندی‌شده، فایل crontab کاربر ریشه (root) را با دستور زیر ویرایش می‌کنیم (برای دسترسی به nmcli و نوشتن در /var/log بهتر است با کاربر روت اجرا شود):

```
sudo crontab -e
```

افزودن وظایف زمان‌بندی‌شده

دو خط زیر را به انتهای فایل crontab اضافه می‌کنیم:

```
# Run the auto-connect script every minute to check the connection
```

```
* * * * * /home/user/scripts/auto-connect.sh
```

Run the auto-connect script once at system boot

@reboot /home/user/scripts/auto-connect.sh

- *****: این بخش به cron می‌گوید که دستور مشخص‌شده را هر دقیقه از هر ساعت، هر روز، هر ماه و هر روز از هفته اجرا کند.
- @reboot: این یک دستور ویژه در cron است که باعث می‌شود دستور مشخص‌شده یک‌بار پس از هر بار بوت شدن (راه‌اندازی مجدد) سیستم اجرا شود.

تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

مکانیزم‌های ارتباط بین‌فرآیندی

۱. FIFO

یک FIFO (مخفف First-In, First-Out) نوعی فایل ویژه در سیستم‌فایل لینوکس است که به عنوان یک کانال ارتباطی عمل می‌کند. این فایل به پراسس‌ها، حتی پراسس‌های غیرمرتبط، اجازه می‌دهد تا با یکدیگر داده تبادل کنند.

- نحوه کارکرد: یک پراسس در یک سر کانال (فایل FIFO) داده می‌نویسد و پراسس دیگر از سر دیگر کانال، داده را می‌خواند. داده‌ها به همان ترتیبی که نوشته شده‌اند، خوانده می‌شوند (اولین ورودی، اولین خروجی).
- هماهنگ‌سازی: سیستم‌عامل به صورت خودکار عملیات خواندن و نوشتن را هماهنگ می‌کند. اگر پراسس گیرنده تلاش کند از یک FIFO خالی بخواند، منتظر (block) می‌ماند تا داده‌ای در آن نوشته شود.

توضیح اسکریپت فرستنده (fifo-sender.sh)

این اسکریپت وظیفه دارد ورودی را از کاربر دریافت کرده و آن را در فایل FIFO بنویسد.

کد اسکریپت:

```
#!/bin/bash
```

```
FIFO_PATH="/tmp/my_fifo"
# Create FIFO if it doesn't exist
if [ ! -p "$FIFO_PATH" ]; then
    mkfifo "$FIFO_PATH"
    chmod 666 "$FIFO_PATH"
fi
# Send messages to FIFO
while true; do
    read -p "Enter message: " message
    if [ "$message" = "exit" ]; then
        break
    fi
    echo "$message" > "$FIFO_PATH"
done
```

تحلیل کد:

- FIFO_PATH="/tmp/my_fifo": یک مسیر استاندارد و قابل دسترس برای فایل FIFO در دایرکتوری /tmp تعریف می‌شود.
- if [! -p "\$FIFO_PATH"]: این شرط بررسی می‌کند که آیا فایلی در مسیر مشخص شده وجود دارد و از نوع FIFO (named pipe) است یا خیر. اگر وجود نداشته باشد، دستورات داخل if اجرا می‌شوند.
- mkfifo "\$FIFO_PATH": این دستور فایل FIFO را در مسیر تعیین‌شده ایجاد می‌کند.
- chmod 666 "\$FIFO_PATH": سطح دسترسی فایل به 666 تغییر داده می‌شود تا همه‌ی کاربران سیستم بتوانند در این فایل بنویسند و از آن بخوانند، که این کار ارتباط بین پراسس‌های با کاربران مختلف را تسهیل می‌کند.

- `while true; do ... done`: یک حلقه بی‌نهایت ایجاد می‌شود تا برنامه به طور مداوم از کاربر ورودی دریافت کند.
- `read -p "Enter message: " message`: برنامه منتظر ورودی کاربر مانده و متن وارد شده را در متغیر `message` ذخیره می‌کند.
- `if [ "$message" = "exit" ]`: اگر کاربر کلمه `exit` را وارد کند، حلقه با دستور `break` متوقف شده و اسکریپت به پایان می‌رسد.
- `echo "$message" > "$FIFO_PATH"`: این خط اصلی‌ترین بخش اسکریپت است. محتوای متغیر `message` با استفاده از ریدایرکشن خروجی (`>`) در فایل FIFO نوشته می‌شود.

توضیح اسکریپت گیرنده (`fifo-receiver.sh`)

این اسکریپت وظیفه دارد به صورت مداوم به فایل FIFO گوش داده و هر داده‌ای که در آن نوشته می‌شود را بخواند و در خروجی نمایش دهد.

کد اسکریپت:

```
#!/bin/bash
FIFO_PATH="/tmp/my_fifo"
# Ensure FIFO exists
if [ ! -p "$FIFO_PATH" ]; then
    echo "FIFO does not exist. Start sender first."
    exit 1
fi
# Receive messages from FIFO
while true; do
    if read line < "$FIFO_PATH"; then
        echo "Received: $line"
    fi
fi
```

done

تحلیل کد:

- `FIFO_PATH="/tmp/my_fifo"`: مسیر فایل FIFO باید دقیقاً مشابه مسیر تعریف شده در اسکریپت فرستنده باشد.
- `if [ ! -p "$FIFO_PATH" ]`: این اسکریپت ابتدا بررسی می‌کند که آیا فایل FIFO وجود دارد یا خیر. اگر وجود نداشته باشد، به این معنی است که فرستنده هنوز اجرا نشده و با نمایش یک پیام خطا از برنامه خارج می‌شود.
- `while true; do ... done`: یک حلقه بی‌نهایت برای گوش دادن دائمی به FIFO ایجاد می‌شود.
- `if read line < "$FIFO_PATH"`: این خط اصلی‌ترین بخش اسکریپت گیرنده است. دستور `read` با استفاده از ریدایرکشن ورودی (`<`) تلاش می‌کند یک خط از فایل FIFO بخواند. این دستور **منتظر (block)** می‌ماند تا زمانی که داده‌ای برای خواندن در FIFO موجود باشد. پس از خواندن، داده در متغیر `line` ذخیره می‌شود.
- `echo "Received: $line"`: پیامی که از FIFO خوانده شده است، در ترمینال نمایش داده می‌شود.

نحوه اجرا و مشاهده نتیجه

برای اجرای صحیح این دو اسکریپت، به دو پنجره ترمینال مجزا نیاز داریم.

۱. سطح دسترسی اجرایی: ابتدا به هر دو اسکریپت دسترسی اجرایی می‌دهیم.

```
chmod +x fifo-sender.sh
```

```
chmod +x fifo-receiver.sh
```

۲. اجرای گیرنده: در ترمینال اول، اسکریپت گیرنده را اجرا می‌کنیم.

```
./fifo-receiver.sh
```

پس از اجرای این دستور، ترمینال در حالت انتظار باقی می‌ماند. این رفتار کاملاً صحیح است، زیرا گیرنده منتظر دریافت داده از FIFO است.

۳. اجرای فرستنده: در ترمینال دوم، اسکریپت فرستنده را اجرا می‌کنیم.

./fifo-sender.sh

برنامه با نمایش پیغام Enter message: از شما ورودی می‌خواهد.

۴. ارسال و دریافت پیام: هر متنی که در ترمینال فرستنده تایپ کرده و Enter را بزنید، بلافاصله در ترمینال گیرنده با پیشوند Received: نمایش داده خواهد شد.

۵. پایان کار: برای خاتمه دادن به فرستنده، کلمه exit را تایپ کنید. اسکریپت گیرنده همچنان فعال باقی می‌ماند و می‌توان فرستنده را مجدداً اجرا کرد.

```
orangepl@orangeizeroplus2h5:~$ ./signal-sender
Enter the PID of the receiver: 3896
Enter the signal number to send: 20
Sent signal 20 to PID 3896
orangepl@orangeizeroplus2h5:~$ ./signal-sender
Enter the PID of the receiver: 3896
Enter the signal number to send: 40
Sent signal 40 to PID 3896
orangepl@orangeizeroplus2h5:~$ ./signal-sender
Enter the PID of the receiver: 3896
Enter the signal number to send: 2
Sent signal 2 to PID 3896
orangepl@orangeizeroplus2h5:~$
```

```
orangepl@orangeizeroplus2h5:~$ ./signal-receiver
Receiver PID: 3896
Received signal: 28 (Window changed)
Received signal: 20 (Stopped)
Received signal: 40 (Real-time signal 6)
Received signal: 28 (Window changed)
^CReceived signal: 2 (Interrupt)
Received signal: 28 (Window changed)
Received signal: 2 (Interrupt)
```

تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

۲. سیگنال‌های لینوکس

سیگنال‌ها یکی از قدیمی‌ترین و اساسی‌ترین مکانیزم‌های ارتباط بین پراسس‌ها (IPC) در سیستم‌عامل‌های یونیکسی مانند لینوکس هستند. یک سیگنال، یک اعلان نرم‌افزاری است که به یک پراسس ارسال می‌شود تا آن را از وقوع یک رویداد مطلع کند. در این تمرین، دو برنامه به زبان C++ به نام‌های signal-sender و signal-receiver ایجاد کرده‌ایم تا نحوه ارسال، دریافت و مدیریت سیگنال‌ها را به صورت عملی نشان دهیم.

توضیح برنامه‌ها

۱. برنامه گیرنده سیگنال (signal-receiver.cpp)

این برنامه طوری طراحی شده است که پس از اجرا، منتظر دریافت سیگنال‌های مختلف از طرف سیستم‌عامل یا پراسس‌های دیگر بماند و هر سیگنال دریافتی را ثبت کند.

تحلیل کد:

```
#include <iostream>
```

```
#include <csignal>
```



```
#include <cstring>
#include <fstream>
#include <unistd.h>

// Open log file globally in append mode
std::ofstream log_file("signal_log.txt", std::ios::app);

// Signal handler function
void signal_handler(int sig) {
    std::string signal_name = ::strsignal(sig); // Get signal name
    std::string message = "Received signal: " + std::to_string(sig) + " (" + signal_name
+ ")\n";
    std::cout << message; // Print to console
    log_file << message; // Log to file
    log_file.flush(); // Ensure log is written immediately
}

int main() {
    // Display the PID of the receiver
    std::cout << "Receiver PID: " << getpid() << std::endl;

    // Register signal handler for all catchable signals
    for (int i = 1; i < NSIG; i++) {
        if (i != SIGKILL && i != SIGSTOP) { // Skip uncatchable signals
            signal(i, signal_handler);
        }
    }

    // Keep the program running indefinitely
```

```

while (true) {
    pause(); // Wait for signals
}

// Unreachable, but included for completeness

log_file.close();

return 0;
}

```

• فایل‌های هدر:

- `<iostream>`: برای چاپ خروجی در کنسول.
- `<csignal>`: شامل تعاریف مربوط به سیگنال‌ها مانند `signal()`، `SIGKILL` و `NSIG`.
- `<cstring>`: برای استفاده از تابع `strsignal()` جهت تبدیل شماره سیگنال به نام آن.
- `<fstream>`: برای کار با فایل‌ها و ثبت لاگ.
- `<unistd.h>`: برای استفاده از توابع سیستمی مانند `getpid()` و `pause()`.

• مدیریت لاگ:

- یک شیء `std::ofstream` به نام `log_file` به صورت سراسری تعریف شده تا فایل `signal_log.txt` را باز کرده و لاگ‌ها را به انتهای آن اضافه کند.

• تابع `signal_handler`:

- این تابع، که به عنوان **Signal Handler** شناخته می‌شود، در هنگام دریافت یک سیگنال اجرا می‌گردد.
- این تابع شماره سیگنال (`sig`) را به عنوان ورودی دریافت می‌کند.
- با استفاده از `strsignal(sig)::` نام متنی سیگنال (مثلاً "Interrupt") استخراج می‌شود.
- یک پیام حاوی شماره و نام سیگنال ساخته شده، در کنسول چاپ شده و همزمان در فایل `signal_log.txt` نیز نوشته می‌شود.

- `log_file.flush()`: این دستور تضمین می‌کند که بافر فایل بلافاصله خالی شده و اطلاعات حتماً روی دیسک نوشته شود، که برای لاگ‌برداری لحظه‌ای بسیار مهم است.

• تابع `main`:

- در ابتدا، شناسه پراسس (PID) برنامه با استفاده از `getpid()` دریافت و چاپ می‌شود. این PID برای برنامه فرستنده جهت ارسال سیگنال ضروری است.
- یک حلقه `for` از ۱ تا `NSIG` (تعداد کل سیگنال‌های تعریف شده در سیستم) اجرا می‌شود تا تابع `signal_handler` را برای تمام سیگنال‌های قابل دریافت (`catchable`) ثبت کند.
- سیگنال‌های `SIGKILL` و `SIGSTOP` از این حلقه مستثنی شده‌اند، زیرا این دو سیگنال قابل مدیریت یا نادیده گرفته شدن نیستند.
- در نهایت، برنامه با دستور `pause()` وارد یک حلقه انتظار بی‌نهایت می‌شود. این دستور مصرف CPU را به حداقل می‌رساند و منتظر می‌ماند تا یک سیگنال دریافت شود.

۲. برنامه فرستنده سیگنال (`signal-sender.cpp`)

این برنامه از کاربر، PID پراسس مقصد و شماره سیگنال مورد نظر را دریافت کرده و آن را به پراسس مشخص شده ارسال می‌کند.

تحلیل کد:

```
#include <iostream>
#include <csignal>
#include <cstdlib>
#include <limits>
int main() {
    int pid, sig;
    // Prompt for PID and validate input
    std::cout << "Enter the PID of the receiver: ";
    while (!(std::cin >> pid) || pid <= 0) {
```

```

std::cout << "Invalid PID. Please enter a positive integer: ";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

// Prompt for signal number and validate input
std::cout << "Enter the signal number to send: ";
while (!(std::cin >> sig) || sig <= 0) {
    std::cout << "Invalid signal number. Please enter a positive integer: ";
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

// Send the signal and check for errors
if (kill(pid, sig) == 0) {
    std::cout << "Sent signal " << sig << " to PID " << pid << std::endl;
} else {
    std::cerr << "Failed to send signal. Check PID and permissions.\n";
}

return 0;
}

```

• دریافت ورودی :

- برنامه از کاربر می‌خواهد تا PID پراسس گیرنده و شماره سیگنال را وارد کند.

○ برای هر دو ورودی، یک حلقه `while` برای اعتبارسنجی ورودی وجود دارد. این حلقه اطمینان می‌دهد که ورودی حتماً یک عدد صحیح مثبت باشد و از کرش کردن برنامه به دلیل ورودی نامعتبر جلوگیری می‌کند.

• ارسال سیگنال :

○ تابع `kill(pid, sig)` که یک فراخوانی سیستمی (System Call) است، برای ارسال سیگنال استفاده می‌شود.

○ اگر تابع `kill` مقدار 0 را برگرداند، به معنی موفقیت‌آمیز بودن ارسال سیگنال است. در غیر این صورت، یک پیام خطا نمایش داده می‌شود که می‌تواند به دلیل نامعتبر بودن PID یا نداشتن مجوز کافی باشد.

نحوه کامپایل و اجرا

کامپایل: برنامه‌ها را با استفاده از کامپایلر `g++` کامپایل می‌کنیم

```
g++ signal-sender.cpp -o sender g++ signal-receiver.cpp -o receiver
```

اجرا: در یک پنجره ترمینال، برنامه گیرنده را اجرا کنید: `./receiver`.

برنامه PID خود را چاپ می‌کند. این عدد را یادداشت کنید. برای مثال Receiver PID: 12345

در یک پنجره ترمینال دیگر، برنامه فرستنده را اجرا کنید: `./sender`.

برنامه فرستنده از شما PID و سپس شماره سیگنال را می‌خواهد PID. یادداشت شده (مثلاً '۱۲۳۴۵') و شماره یک سیگنال (مثلاً '۲' برای 'SIGINT') را وارد کنید.

مشاهده نتیجه: بلافاصله پس از ارسال، در ترمینال گیرنده پیامی مبنی بر دریافت سیگنال چاپ می‌شود. همچنین همین پیام در فایل `signal_log.txt` نیز ثبت می‌گردد.

پاسخ به سوالات کلیدی

۱. تفاوت بین سیگنال‌های SIGINT و SIGKILL چیست؟

SIGINT (Signal Interrupt, signal 2) و SIGKILL (Signal Kill, signal 9) هر دو به طور پیش‌فرض باعث خاتمه یافتن یک پراسس می‌شوند، اما تفاوت اساسی آن‌ها در نحوه مدیریت آن‌ها توسط پراسس است.

- **SIGINT**: این سیگنال یک "درخواست مودبانه" برای خاتمه دادن به برنامه است. یک پراسس می‌تواند این سیگنال را دریافت کرده (catch)، آن را نادیده بگیرد (ignore)، یا یک تابع مدیریت‌کننده (handler) برای آن تعریف کند تا کارهای پاک‌سازی (مانند بستن فایل‌ها یا اتصالات شبکه) را قبل از خروج انجام دهد. این همان سیگنالی است که با فشردن کلیدهای Ctrl+C در ترمینال ارسال می‌شود.

- **SIGKILL**: این سیگنال یک "دستور قطعی" برای خاتمه دادن فوری به برنامه است. این سیگنال هرگز توسط پراسس قابل دریافت، مدیریت یا نادیده گرفتن نیست. وقتی کرنل این سیگنال را به یک پراسس ارسال می‌کند، آن پراسس بدون هیچ فرصتی برای پاک‌سازی، فوراً متوقف می‌شود. این سیگنال به عنوان آخرین راه‌حل برای از بین بردن پراسس‌های هنگ کرده یا سرکش استفاده می‌شود.

۲. کدام سیگنال‌ها نمی‌توانند توسط یک پراسس مدیریت شوند؟

اکثر سیگنال‌ها قابل مدیریت هستند، اما دو سیگنال استثنای بسیار مهم وجود دارد که هیچ پراسسی نمی‌تواند آن‌ها را مدیریت کند:

- **SIGKILL (Signal 9)**

- **SIGSTOP (Signal 19)**

این دو سیگنال به این دلیل غیرقابل مدیریت طراحی شده‌اند که یک راه تضمینی برای سیستم‌عامل و مدیر سیستم فراهم کنند تا بتوانند کنترل هر پراسسی را، حتی اگر به درستی کار نکند، در دست بگیرند. SIGKILL پراسس را از بین می‌برد و SIGSTOP آن را متوقف (pause) می‌کند.

۳. چند سیگنال در لینوکس تعریف شده‌اند؟

تعداد دقیق سیگنال‌ها می‌تواند بسته به معماری پردازنده و نسخه کرنل لینوکس کمی متفاوت باشد. اما به طور کلی در سیستم‌های مدرن لینوکس:

- سیگنال‌های استاندارد: ۳۱ سیگنال از شماره ۱ تا ۳۱.

- سیگنال‌های بی‌درنگ (Real-time signals): سیگنال‌های شماره ۳۲ تا ۶۴.

بنابراین، روی هم رفته ۶۴ سیگنال در اکثر سیستم‌های لینوکسی امروزی تعریف شده است. می‌توان لیست کامل سیگنال‌ها را با دستور `kill -l` در ترمینال مشاهده کرد. ثابت `NSIG` در فایل هدر `<signal.h>` نیز حد بالای شماره سیگنال‌ها را مشخص می‌کند.

تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

۳. حافظه اشتراکی

حافظه اشتراکی (Shared Memory) یکی از سریع‌ترین روش‌های ارتباط بین پراسس‌ها (IPC) در لینوکس است. در این روش، یک قطعه از حافظه RAM به فضای آدرس‌دهی مجازی چندین پراسس نگاشت (map) می‌شود. این کار به پراسس‌ها اجازه می‌دهد تا با خواندن و نوشتن مستقیم در این حافظه مشترک، داده‌ها را با سرعت بسیار بالا و بدون دخالت هسته (Kernel) در تبادل داده، با یکدیگر به اشتراک بگذارند.

در این تمرین، دو برنامه به زبان C++ با نام‌های `shm-reader` و `shm-writer` پیاده‌سازی شده‌اند که از حافظه اشتراکی برای پیاده‌سازی یک بافر دایره‌ای (Circular Buffer) جهت تبادل داده استفاده می‌کنند.

توضیح برنامه‌ها

۱. برنامه نویسنده (`shm-writer.cpp`)

این برنامه یک قطعه حافظه اشتراکی ایجاد می‌کند، از کاربر ورودی می‌گیرد و آن را در یک بافر دایره‌ای ۱۰ کاراکتری در آن حافظه می‌نویسد.

تحلیل کد:

```
#include <iostream>
```

```
#include <fcntl.h>
```

```
#include <sys/stat.h>
```

```
#include <sys/mman.h>
```

```
#include <unistd.h>
```

```
#include <cstring>
```

```
#include <cerrno>
```

```
#include <string>

#include <algorithm>

#define SHM_SIZE 10

const char* SHM_NAME = "/my_shm";

int main() {

    const size_t meta_size = sizeof(size_t) * 2;

    const size_t total_size = meta_size + SHM_SIZE;

    bool created = false;

    int fd = shm_open(SHM_NAME, O_CREAT | O_EXCL | O_RDWR, 0666);

    if (fd >= 0) {

        created = true;

    } else if (errno == EEXIST) {

        fd = shm_open(SHM_NAME, O_RDWR, 0);

        if (fd < 0) {

            std::cerr << "shm_open existing: " << strerror(errno) << "\n";

            return 1;

        }

    } else {

        std::cerr << "shm_open create: " << strerror(errno) << "\n";

        return 1;

    }

    if (created) {

        if (ftruncate(fd, total_size) < 0) {
```



```
std::cerr << "ftruncate: " << strerror(errno) << "\n";
```

```
close(fd);
```

```
shm_unlink(SHM_NAME);
```

```
return 1;
```

```
}
```

```
}
```

```
void* addr = mmap(nullptr, total_size,
```

```
    PROT_READ | PROT_WRITE,
```

```
    MAP_SHARED,
```

```
    fd, 0);
```

```
if (addr == MAP_FAILED) {
```

```
    std::cerr << "mmap: " << strerror(errno) << "\n";
```

```
    close(fd);
```

```
    if (created) shm_unlink(SHM_NAME);
```

```
    return 1;
```

```
}
```

```
close(fd);
```

```
size_t* write_index_shm = static_cast<size_t*>(addr);
```

```
size_t* last_len_shm  = static_cast<size_t*>(addr) + 1;
```

```
char* buffer          = reinterpret_cast<char*>(addr) + meta_size;
```

```
if (created) {
```

```
    *write_index_shm = 0;
```

```
    *last_len_shm    = 0;
```

```
    std::memset(buffer, ' ', SHM_SIZE);
```

```

    }

    std::cout << "Writer started. Type a line, press ENTER to write.\n"
        << "The circular buffer holds " << SHM_SIZE << " chars.\n"
        << "Press Ctrl+D or type an empty line then Enter to quit.\n";

    std::string line;

    while (std::getline(std::cin, line) && !line.empty()) {
        size_t current_input_len = line.size();

        for (size_t i = 0; i < current_input_len; ++i) {
            buffer[*write_index_shm] = line[i];

            *write_index_shm = (*write_index_shm + 1) % SHM_SIZE;
        }

        *last_len_shm = std::min(current_input_len, static_cast<size_t>(SHM_SIZE));

        std::cout << "[Wrote " << current_input_len
            << " chars. Next write pos (write_index)=" << *write_index_shm << "]\n";
    }

    if (munmap(addr, total_size) == -1) {
        std::cerr << "munmap: " << strerror(errno) << "\n";
    }

    std::cout << "Writer finished.\n";

    return 0;
}

```

• فراخوانی‌های سیستمی کلیدی :

- shm_open(): برای ایجاد یک شیء حافظه اشتراکی جدید در سیستم فایل مجازی با نام ./my_shm. فلگ O_CREAT | O_EXCL تضمین می‌کند که اگر شیء از قبل وجود نداشته باشد، ایجاد شود. اگر وجود داشته باشد (EEXIST)، برنامه آن را در حالت خواندن و نوشتن باز می‌کند.

- ftruncate(): پس از ایجاد حافظه اشتراکی، این تابع برای تنظیم اندازه دقیق آن استفاده می‌شود.

- mmap(): مهم‌ترین تابع که شیء حافظه اشتراکی را به فضای آدرس‌دهی پراسس فعلی نگاشت می‌دهد و یک اشاره‌گر (void*) به ابتدای آن برمی‌گرداند. از این پس، می‌توان با این اشاره‌گر مانند یک آرایه معمولی در حافظه کار کرد.

- **ساختار حافظه اشتراکی:** این برنامه برای مدیریت بافر دایره‌ای، از یک ساختار هوشمندانه در حافظه اشتراکی استفاده می‌کند. حافظه تنها شامل داده‌ها نیست، بلکه شامل **فرا داده (metadata)** نیز می‌باشد:

- write_index_shm *size_t: یک اشاره‌گر به اندیسی که عمل نوشتن بعدی باید در آنجا انجام شود.

- last_len_shm *size_t: یک اشاره‌گر که طول آخرین ورودی نوشته شده را نگهداری می‌کند.

- buffer *char: اشاره‌گر به ابتدای بافر ۱۰ کاراکتری که داده‌های کاربر در آن ذخیره می‌شود.

- **منطق بافر دایره‌ای:**

- یک حلقه while ورودی را خط به خط از کاربر دریافت می‌کند.

- کاراکترهای ورودی یک به یک به buffer نوشته می‌شوند.

- پس از نوشتن هر کاراکتر، write_index_shm با استفاده از عملیات باقی‌مانده (% SHM_SIZE) به‌روزرسانی می‌شود تا به خانه بعدی اشاره کند و در صورت رسیدن به انتهای بافر، به ابتدای آن بازگردد.

- **پاک‌سازی:**

- `munmap()`: قبل از خروج، نگاشت حافظه را از فضای آدرس‌دهی پراسس حذف می‌کند.
- **نکته مهم:** برنامه `shm_unlink(SHM_NAME)` را فراخوانی نمی‌کند. این تابع برای حذف کامل شیء حافظه اشتراکی از سیستم ضروری است. بدون آن، حافظه حتی پس از پایان کار هر دو برنامه در سیستم باقی می‌ماند. این کار معمولاً باید توسط نویسنده پس از اتمام کار یا توسط یک اسکریپت پاک‌سازی انجام شود.

۲. برنامه خواننده (`shm-reader.cpp`)

این برنامه به حافظه اشتراکی ایجاد شده توسط نویسنده متصل شده و محتویات خام بافر و فراداده آن را به درخواست کاربر نمایش می‌دهد.

تحلیل کد:

```
#include <iostream>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>
#include <cerrno>
#include <cstring>
#include <string>
#define SHM_SIZE 10
const char* SHM_NAME = "/my_shm";

int main() {
    const size_t meta_size = sizeof(size_t) * 2;
    const size_t total_size = meta_size + SHM_SIZE;
```

```

int fd = shm_open(SHM_NAME, O_RDONLY, 0);
if (fd < 0) {
    std::cerr << "shm_open: " << strerror(errno) << "\n";
    std::cerr << "Ensure the writer process has been run first to create the shared
memory.\n";
    return 1;
}

void* addr = mmap(nullptr, total_size,
    PROT_READ,
    MAP_SHARED,
    fd, 0);
if (addr == MAP_FAILED) {
    std::cerr << "mmap: " << strerror(errno) << "\n";
    close(fd);
    return 1;
}

close(fd);

size_t* write_index_ptr = static_cast<size_t*>(addr);
size_t* last_len_ptr = static_cast<size_t*>(addr) + 1;
char* buffer = reinterpret_cast<char*>(addr) + meta_size;

std::cout << "Reader started. Press ENTER to dump raw shared memory
buffer.\n"
    << "Press Ctrl+D to quit.\n";

while (std::cin.get() == '\n') {

```

```

size_t current_write_index = *write_index_ptr;
size_t current_last_len = *last_len_ptr;

std::cout << "Buffer: [";

for (size_t i = 0; i < SHM_SIZE; ++i) {

    std::cout << buffer[i];

}

std::cout << "]" (last_len=" << current_last_len

    << ", write_index=" << current_write_index << ")\n";

}

if (munmap(addr, total_size) == -1) {

    std::cerr << "munmap: " << strerror(errno) << "\n";

}

std::cout << "Reader finished.\n";

return 0;

}

```

- shm_open(): این بار، حافظه اشتراکی موجود با فلگ O_RDONLY (فقط خواندنی) باز می‌شود.
- mmap(): حافظه با دسترسی فقط خواندنی (PROT_READ) به فضای آدرس‌دهی خواننده نگاشت می‌شود.
- خواندن از حافظه:
 - این برنامه نیز از همان ساختار حافظه (فراداده + بافر) برای تفسیر صحیح داده‌ها استفاده می‌کند.
 - یک حلقه while منتظر می‌ماند تا کاربر کلید Enter را فشار دهد.
 - پس از آن، مقادیر فعلی write_index و last_len را مستقیماً از حافظه اشتراکی خوانده و کل محتوای خام بافر را به همراه این مقادیر نمایش می‌دهد.

• پاک‌سازی:

○ در انتها، `munmap()` برای حذف نگاشت حافظه فراخوانی می‌شود.

نحوه کامپایل و اجرا

کامپایل: برای کامپایل این برنامه‌ها باید کتابخانه `real-time (rt)` را لینک کنیم.

```
g++ shm-writer.cpp -o writer -lrt g++ shm-reader.cpp -o reader -lrt
```

اجرا: در ترمینال اول، برنامه نویسنده را اجرا کنید. این برنامه حافظه اشتراکی را ایجاد می‌کند `/writer`. در ترمینال دوم، برنامه خواننده را اجرا کنید `/reader`. در ترمینال نویسنده، یک متن (مثلاً Hello) وارد کرده و `Enter` را بزنید. در ترمینال خواننده، `Enter` را بزنید. خواهید دید که محتوای بافر و مقادیر فراداده به‌روز شده‌اند. اگر در نویسنده متنی طولانی‌تر از ۱۰ کاراکتر وارد کنید (مثلاً 1234567890ABCDE)، خواهید دید که کاراکترهای جدید جایگزین کاراکترهای ابتدایی بافر می‌شوند که این همان رفتار بافر دایره‌ای است.

```
orangepi@orangeizeroplus2h5:~$ ./shm-writer
Writer started. Type a line, press ENTER to write.
The circular buffer holds 10 chars.
Press Ctrl+D or type an empty line then Enter to quit.
123456
[Wrote 6 chars. Next write pos (write_index)=6]
123456
[Wrote 6 chars. Next write pos (write_index)=2]
1111111111111111
[Wrote 16 chars. Next write pos (write_index)=8]
11
[Wrote 2 chars. Next write pos (write_index)=0]
asgwpgnurpegwprguinessg
[Wrote 25 chars. Next write pos (write_index)=5]
```

```
orangepi@orangeizeroplus2h5:~$ ./shm-reader
Reader started. Press ENTER to dump raw shared memory buffer.
Press Ctrl+D to quit.
Buffer: [          ] (last_len=0, write_index=0)
Buffer: [123456    ] (last_len=6, write_index=6)
Buffer: [5634561234] (last_len=6, write_index=2)
Buffer: [1111111111] (last_len=10, write_index=8)
Buffer: [nessgprgui] (last_len=10, write_index=5)
```

بخش سوم: پاسخ به سوال کلیدی

تفاوت‌های کلیدی بین `FIFO` و حافظه اشتراکی برای `IPC` چیست؟

ویژگی	FIFO (Named Pipe)	حافظه اشتراکی (Shared Memory)
سرعت	کندتر. داده‌ها باید دو بار کپی شوند: ۱. از پراسس نویسنده به هسته (Kernel).	بسیار سریع. سریع‌ترین روش <code>IPC</code> است، زیرا پس از نگاشت اولیه، هیچ دخالتی از سوی

	۲. از هسته به پراسس خواننده.	هسته برای تبادل داده وجود ندارد و کپی اضافه‌ای صورت نمی‌گیرد.
همگام‌سازی (Synchronization)	داخلی و خودکار. هسته عملیات را مدیریت می‌کند. اگر خواننده از FIFO خالی بخواند، منتظر (block) می‌ماند. این کار استفاده از آن را ساده‌تر می‌کند.	بدون همگام‌سازی داخلی. این بزرگترین چالش است. پراسس‌ها باید خودشان مکانیزم‌های همگام‌سازی (مانند semaphore، mutex یا فراداده سفارشی) را پیاده‌سازی کنند تا از تداخل و race condition جلوگیری شود.
ساختار داده	جریان بایتی بدون ساختار (Unstructured Byte Stream). داده‌ها به صورت یک جریان پیوسته از بایت‌ها هستند و مرز مشخصی بین پیام‌ها وجود ندارد، مگر اینکه برنامه‌ها پروتکلی برای آن تعریف کنند.	کاملاً منعطف و ساختاریافته. پراسس‌ها می‌توانند هر نوع ساختار داده‌ای (مانند struct، آرایه، کلاس) را روی حافظه مشترک اعمال کنند.
سهولت استفاده	ساده. کار با آن بسیار شبیه به کار با فایل‌های عادی است.	پیچیده‌تر. به دلیل نیاز به مدیریت همگام‌سازی دستی، پیاده‌سازی آن دشوارتر و مستعد خطا است.

خلاصه :

- از FIFO زمانی استفاده کنید که سادگی پیاده‌سازی و همگام‌سازی خودکار مهم‌تر از سرعت است.
 - از حافظه اشتراکی زمانی استفاده کنید که به حداکثر سرعت ممکن برای تبادل حجم بالایی از داده نیاز دارید و آماده‌اید پیچیدگی مدیریت همگام‌سازی را بپذیرید.
- تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

وب سرور پایه

در این تمرین، یک وب سرور بسیار ساده و پایه را با استفاده از یک اسکریپت شل (bash) و ابزار قدرتمند nc (netcat) پیاده سازی می کنیم. سپس برای مدیریت این اسکریپت به عنوان یک سرویس پایدار در سیستم عامل لینوکس، یک واحد (unit) سرویس systemd ایجاد می کنیم. این سرویس تضمین می کند که وب سرور ما به طور خودکار پس از راه اندازی سیستم و برقراری اتصال شبکه اجرا شده و در صورت بروز خطا، مجدداً راه اندازی شود.

اسکریپت وب سرور (simple-web-server.sh)

این اسکریپت قلب وب سرور ماست و وظیفه گوش دادن به درخواست های ورودی و ارسال پاسخ مناسب را بر عهده دارد.

```
#!/usr/bin/env bash
```

```
PORT=8080
```

```
echo "Listening on port $PORT"
```

```
echo
```

```
while true; do
```

```
BODY="Hello I,m Meysam Elhambakhsh $(date '+%Y-%m-%d %H:%M:%S')"
```

```
echo -e "HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nConnection: close\r\n\r\n$BODY" \
```

```
| nc -l -p "$PORT" -q 1 >/dev/null 2>&1
```

```
echo
```

```
echo "$BODY"
```

```
done
```

تحلیل کد:

- PORT=8080: در این خط، پورت سرور تعریف شده است .

○ نکته: در صورت سوال، استفاده از پورت ۸۰ خواسته شده است. برای استفاده از پورت ۸۰ نیاز به دسترسی **root** است. پورت ۸۰۸۰ یک جایگزین رایج است که به دسترسی **root** نیاز ندارد. برای استفاده از پورت ۸۰، کافی است این مقدار را به ۸۰ تغییر داده و اسکریپت را با **sudo** اجرا کنید.

- **while true; do ... done**: این اسکریپت در یک حلقه بی‌نهایت اجرا می‌شود تا بتواند به طور مداوم به درخواست‌های ورودی گوش دهد و پس از پاسخ به هر درخواست، برای درخواست بعدی آماده شود.
- **BODY="..."**: در این بخش، بدنه پاسخ **HTTP** ساخته می‌شود که شامل یک پیام خوشامدگویی و تاریخ و زمان فعلی سیستم است. تابع **date** این اطلاعات را به صورت پویا تولید می‌کند.
- **nc ... | echo -e "..."**: این خط اصلی‌ترین بخش سرور است .

○ **echo -e**: با استفاده از این دستور، یک پاسخ کامل **HTTP** ساخته می‌شود که شامل خط وضعیت (**HTTP/1.1 200 OK**)، هدرها (**Content-Type** و **Connection**) و بدنه (**\$BODY**) است. **\r\n** کاراکترهای استاندارد پایان خط در پروتکل **HTTP** هستند.

○ **nc -l -p "\$PORT" -q 1**: خروجی دستور **echo** از طریق یک **pipe** به ورودی ابزار **nc** (**netcat**) ارسال می‌شود. **nc** با این آپشن‌ها یک سرور ایجاد می‌کند:

- **-l**: حالت گوش دادن. (**Listen**)
- **-p "\$PORT"**: پورتنی که باید روی آن گوش دهد.
- **-q 1**: پس از دریافت یک درخواست و ارسال پاسخ، ۱ ثانیه منتظر مانده و سپس اتصال را می‌بندد تا برای درخواست بعدی آماده شود.

• **echo "\$BODY"**: این خط، همانطور که در صورت سوال خواسته شده، بدنه پاسخ را در ترمینالی که اسکریپت در آن اجرا شده نیز چاپ می‌کند.

سرویس (Systemd (simple-web-server.service

برای اینکه اسکریپت ما به یک سرویس واقعی تبدیل شود که به صورت خودکار مدیریت شود، از **systemd** استفاده می‌کنیم.

فایل سرویس:

[Unit]

Description=Simple netcat-based HTTP Server

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

ExecStart=/home/orangepi/simple-web-server.sh

Restart=on-failure

RestartSec=5

StandardOutput=syslog

StandardError=syslog

SyslogIdentifier=simple-web-server

[Install]

WantedBy=multi-user.target

تحلیل فایل سرویس:

- بخش [Unit]: این بخش اطلاعات کلی و وابستگی‌های سرویس را مشخص می‌کند .
 - Description: یک توضیح کوتاه برای سرویس.
 - After=network-online.target: یک وابستگی مهم که به systemd می‌گوید این سرویس را فقط پس از اینکه شبکه به طور کامل آنلاین و متصل شد، اجرا کند.
- بخش [Service]: این بخش نحوه اجرای سرویس را تعریف می‌کند .

○ `ExecStart=/home/orangepi/simple-web-server.sh`: مسیر مطلق به اسکریپت وب سرور. این مسیر باید در سیستم شما صحیح باشد.

○ `Restart=on-failure`: یک قابلیت کلیدی برای پایداری سرویس. اگر اسکریپت با خطا (کد خروجی غیر صفر) متوقف شود، `systemd` به طور خودکار آن را مجدداً اجرا می‌کند.

○ `RestartSec=5`: تأخیر ۵ ثانیه‌ای قبل از تلاش برای راه‌اندازی مجدد.

○ `StandardOutput=syslog` و `StandardError=syslog`: تمام خروجی‌های استاندارد و خطاهای اسکریپت به جای نمایش در ترمینال، به لاگ سیستم (`journald`) ارسال می‌شوند.

○ `SyslogIdentifier=simple-web-server`: یک برچسب مشخص برای لاگ‌های این سرویس تا به راحتی قابل فیلتر کردن باشند.

• بخش **[Install]**: این بخش مشخص می‌کند که سرویس چگونه باید فعال شود .

○ `WantedBy=multi-user.target`: این خط به `systemd` می‌گوید که این سرویس باید هنگام بوت شدن سیستم و رسیدن به حالت چندکاربره (حالت استاندارد سیستم‌های سرور) فعال شود.

نحوه استقرار و مدیریت سرویس

۱. قراردادن اسکریپت: اسکریپت `simple-web-server.sh` را در مسیر مشخص شده در فایل سرویس (مثلاً `/home/orangepi/`) قرار داده و به آن دسترسی اجرایی بدهید:

```
chmod +x /home/orangepi/simple-web-server.sh
```

۲. قراردادن فایل سرویس: فایل `simple-web-server.service` را در مسیر سرویس‌های `systemd` کپی کنید:

```
sudo cp simple-web-server.service /etc/systemd/system/
```

۳. بارگذاری مجدد **Systemd**: به `systemd` اطلاع دهید که یک فایل سرویس جدید اضافه شده است:

```
sudo systemctl daemon-reload
```

۴. فعال‌سازی سرویس: برای اینکه سرویس هنگام بوت شدن سیستم به طور خودکار اجرا شود، آن را فعال (`enable`) کنید:

```
sudo systemctl enable simple-web-server.service
```

۵. اجرای دستی سرویس: برای اجرای سرویس به صورت دستی (بدون نیاز به ری بوت):

```
sudo systemctl start simple-web-server.service
```

۶. بررسی وضعیت: برای مشاهده وضعیت فعلی سرویس (فعال بودن، PID، آخرین لاگ‌ها):

```
sudo systemctl status simple-web-server.service
```

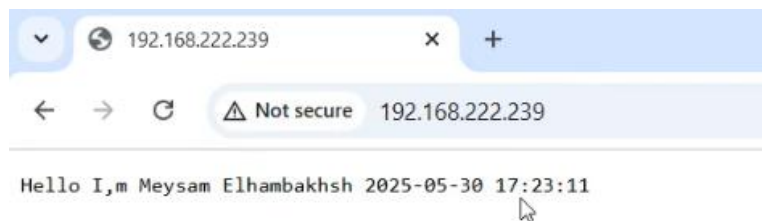
۷. مشاهده لاگ‌ها: برای مشاهده زنده لاگ‌های سرویس:

```
journalctl -u simple-web-server.service -f
```

۸. تست وب سرور: با استفاده از یک مرورگر وب یا ابزار curl در ترمینال، به آدرس IP برد خود با پورت مشخص شده (مثلاً ۸۰) یک درخواست ارسال کنید:

```
curl http://<IP_ADDRESS>:80
```

در پاسخ باید پیام خوشامدگویی و تاریخ و زمان فعلی را دریافت کنید.



تمامی موارد خواسته شده در این بخش پیاده شده است و میتوانید در ویدیو مربوطه تست آن را مشاهده فرمایید.

References

1. runtimerec.com, accessed June 1, 2025, https://runtimerec.com/wp-content/uploads/2024/10/real-time-performance-in-linux-harnessing-preempt-rt-for-embedded-systems_67219ae1.pdf
2. Real-Time OS: The Total Guide to Why and How to Use a Real-Time Operating System, accessed June 1, 2025, <https://www.suse.com/c/what-is-a-real-time-operating-system/>
3. linux - Disadvantage(s) of preempt_rt - Stack Overflow, accessed June 1, 2025, <https://stackoverflow.com/questions/10042550/disadvantages-of-preempt-rt>
4. PREEMPT_RT - Wikipedia, accessed June 1, 2025, https://en.wikipedia.org/wiki/PREEMPT_RT
5. Real-Time Linux | Toradex Developer Center, accessed June 1, 2025, <https://developer.toradex.com/software/real-time/real-time-linux/>
6. Real-time Linux vs RTOS – Part II | Ubuntu, accessed June 1, 2025, <https://ubuntu.com/blog/real-time-linux-vs-rtos-2>
7. Performance Assessment of Linux Kernels with PREEMPT_RT on ..., accessed June 1, 2025, <https://www.mdpi.com/2079-9292/10/11/1331>
8. I Dunno. "The main aim of the PREEMPT_RT patch is to minimize the amount of kern... | Hacker News, accessed June 1, 2025, <https://news.ycombinator.com/item?id=41695152>
9. A Preliminary Assessment of the real-time capabilities of Real-Time Linux on Raspberry Pi 5 - Antonio Paolillo, accessed June 1, 2025, https://antonio.paolillo.be/publications/workshops/ecrtsOspert2024_dewit_rtlinux_paper.pdf
10. SCHED_DEADLINE - Wikipedia, accessed June 1, 2025, https://en.wikipedia.org/wiki/SCHED_DEADLINE
11. Chapter 5. Priorities and Policies | Reference Guide | Red Hat ..., accessed June 1, 2025, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_for_real_time/7/html/reference_guide/chap-priorities_and_policies
12. Chapter 7. Scheduling policies for RHEL for Real Time - Red Hat Documentation, accessed June 1, 2025, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_for_real_time/9/html/understanding_rhel_for_real_time/assembly_scheduling-policies-for-rhel-for-real-time_understanding-rhel-for-real-time-core-concepts
13. Real-Time Scheduling on Linux — ECI documentation, accessed June 1, 2025, https://eci.intel.com/docs/3.0.2/development/rt_scheduling.html

14. Linux SCHED_OTHER, SCHED_FIFO and SCHED_RR ..., accessed June 1, 2025, <https://stackoverflow.com/questions/9392415/linux-sched-other-sched-fifo-and-sched-rr-differences>
15. Achieving Hard Real-Time Performance on Linux with RT Preempt and C Language | IT trip, accessed June 1, 2025, <https://en.ittrip.xyz/c-language/rt-preempt-c-real-time>
16. Real-Time Scheduling on Linux — ECI documentation, accessed June 1, 2025, https://eci.intel.com/docs/3.3/development/performance/rt_scheduling.html
17. UBI File System — The Linux Kernel documentation, accessed June 1, 2025, <https://www.kernel.org/doc/html/v5.15/filesystems/ubifs.html>
18. Managing flash storage with Linux - Bootlin, accessed June 1, 2025, <https://bootlin.com/blog/managing-flash-storage-with-linux/>
19. [1208.6391] On Benchmarking Embedded Linux Flash File Systems, accessed June 1, 2025, <https://ar5iv.labs.arxiv.org/html/1208.6391>
20. UBI File System — The Linux Kernel documentation, accessed June 1, 2025, <https://www.kernel.org/doc/html/v5.19/filesystems/ubifs.html>
21. linux - Algorithm used in JFFS2 - Stack Overflow, accessed June 1, 2025, <https://stackoverflow.com/questions/10754001/algorithm-used-in-jffs2>
22. Wear leveling - Wikipedia, accessed June 1, 2025, https://en.wikipedia.org/wiki/Wear_leveling
23. Embedded Linux vs Desktop Linux - EMAC inc., accessed June 1, 2025, https://shop.emacinc.com/wp-content/uploads/2013/02/OE5_whitepaper.pdf
24. Prevent data corruption on ext4/Linux drive on power loss - Server Fault, accessed June 1, 2025, <https://serverfault.com/questions/318104/prevent-data-corruption-on-ext4-linux-drive-on-power-loss>
25. "safe" ext4 configuration for systems running unattended - Server Fault, accessed June 1, 2025, <https://serverfault.com/questions/356507/safe-ext4-configuration-for-systems-running-unattended>
26. How to deal with power outages in memory products that use file systems? - Semiconductor business -Macnica, accessed June 1, 2025, <https://www.macnica.co.jp/en/business/semiconductor/articles/tuxera/144098/>
27. JFFS2 versus YAFFS2 and UBIFS - Google Groups, accessed June 1, 2025, <https://groups.google.com/g/comp.os.linux.embedded/c/sFoyCQoE1bE>

28. Embedded Linux (پیشرونیك) - بخش دوم, accessed June 1, 2025, <https://pishronic.ir/%D9%85%D9%82%D8%AF%D9%85%D9%87-%D8%A7%DB%8C-%D8%A8%D8%B1-embedded-linux-%D8%A8%D8%AE%D8%B4-%D8%AF%D9%88%D9%85/>
29. owasp.org, accessed June 1, 2025, <https://owasp.org/www-chapter-toronto/assets/slides/2019-12-11-OWASP-IoT-Top-10---Introduction-and-Root-Causes.pdf>
30. OWASP IoT Top 10 Vulnerabilities (2025 Updated) - Wattlecorp Cybersecurity Labs, accessed June 1, 2025, <https://www.wattlecorp.com/owasp-iot-top-10/>
31. CVE-2025-32433: Unauthenticated RCE Vulnerability in Erlang ..., accessed June 1, 2025, <https://www.cybereason.com/blog/rce-vulnerability-erlang-otp>
32. Understanding Linux Kernel Security for Embedded Systems, accessed June 1, 2025, <https://tuxcare.com/blog/embedded-linux-security/>
33. Embedded Linux for Optimized System Performance | Lenovo US, accessed June 1, 2025, <https://www.lenovo.com/us/en/glossary/embedded-linux/>
34. Using Device Tree Overlays, example on BeagleBone Cape add-on ..., accessed June 1, 2025, <https://www.beagleboard.org/blog/2022-02-15-using-device-tree-overlays-example-on-beaglebone-cape-add-on-boards>
35. Use Device Tree Overlays to Patch Your Device Tree - Digi International, accessed June 1, 2025, <https://www.digi.com/resources/examples-guides/use-device-tree-overlays-to-patch-your-device-tree>
36. Mastering Linux Device Trees in Embedded Systems, accessed June 1, 2025, <https://octavosystems.com/linux-device-trees-embedded-systems/>
37. Mastering Device Tree Overlays in Raspberry Pi: A Guide for ..., accessed June 1, 2025, <https://hubtronics.in/learn/mastering-device-tree-overlays-rpi>
38. First Steps with Device Tree Overlays | Toradex Developer Center, accessed June 1, 2025, <https://developer.toradex.com/software/linux-resources/device-tree/first-steps-with-device-tree-overlays/>
39. Devicetree Overlay Notes — The Linux Kernel documentation, accessed June 1, 2025, <https://docs.kernel.org/devicetree/overlay-notes.html>
40. Enabling new hardware on Raspberry Pi with Device Tree Overlays - Bootlin, accessed June 1, 2025, <https://bootlin.com/blog/enabling-new-hardware-on-raspberry-pi-with-device-tree-overlays/>

41. Linux kernel - Wikipedia, accessed June 1, 2025, https://en.wikipedia.org/wiki/Linux_kernel
42. Linux kernel assembly and logic - Stack Overflow, accessed June 1, 2025, <https://stackoverflow.com/questions/22122887/linux-kernel-assembly-and-logic>
43. Assembly in the Linux Kernel : r/kernel - Reddit, accessed June 1, 2025, https://www.reddit.com/r/kernel/comments/2uluk2/assembly_in_the_linux_kernel/
44. 16. Physical Layout of the Kernel Source - Linux Device Drivers ..., accessed June 1, 2025, <https://www.oreilly.com/library/view/linux-device-drivers/0596000081/ch16.html>
45. Booting process of Linux - Wikipedia, accessed June 1, 2025, https://en.wikipedia.org/wiki/Booting_process_of_Linux
46. [www.kernel.org](https://www.kernel.org/doc/html/v5.6/asm-annotations.html#:~:text=SYM_CODE_START%20%2C%20and%20similar.-,Rationale,or%20even%20specifying%20their%20length.), accessed June 1, 2025, https://www.kernel.org/doc/html/v5.6/asm-annotations.html#:~:text=SYM_CODE_START%20%2C%20and%20similar.-,Rationale,or%20even%20specifying%20their%20length.